

ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS

ENSAM MEKNÈS — Maroc

Filière IATD SI — Intelligence Artificielle et Technologies des Données : Systèmes Industriels
Section 8

RAPPORT DE TRAVAUX PRATIQUE 3

TP3 — Atelier de Réalité Augmentée
Unity 3D & Vuforia Engine
Compte rendu de réalisation et d'innovation

Application AR
2ème année — Cycle Ingénieur

Par :

DJERI ALASSANI OUBENOUPOU

Encadrants : Mr Ajebli Ahmed | Mr Maalouf Imad

Element: Virtual and Augmented Reality

Date: 24/04/26

Table des matières

Cliquer-droit sur la table puis « Mettre à jour les champs » pour générer la pagination automatique dans Word.

Table des matières	1
1. Introduction	5
1.1. Contexte de l'atelier	5
1.2. Présentation de Vuforia Engine	5
1.3. Objectifs du compte rendu	5
PARTIE I — Réalisation du TP de base.....	6
2. Préparation de l'environnement de travail.....	6
2.1. Vérification des prérequis	6
2.2. Téléchargement du package Vuforia Engine	6
2.3. Création du projet Unity et import du package.....	7
3. Configuration de la licence Vuforia.....	9
3.1. Génération de la clé de licence	9
3.2. Insertion de la clé dans Unity.....	9
4. Configuration de la scène AR	11
4.1. Suppression de la Main Camera et ajout de l'AR Camera	11
4.2. Ajout de l'Image Target.....	12
5. Intégration du contenu 3D animé.....	13
5.1. Récupération d'un personnage et d'une animation sur Mixamo	13
5.2. Intégration du personnage dans Unity	14
5.3. Configuration de l'animation en boucle.....	16
6. Phase de test dans l'éditeur Unity	19
6.1. Test avec la webcam du laptop.....	19
7. Création du Build Android (APK).....	19
7.1. Prérequis pour le build Android.....	19
7.2. Paramétrage du projet pour Android	20
7.3. Configuration cruciale du XR Plug-in Management.....	20
7.4. Lancement du build et installation sur smartphone.....	21
8. Conclusion de la partie 1.....	21
PARTIE II — Travail d'innovation et d'approfondissement.....	23
9. Démarche et choix du projet d'innovation	23
9.1. Motivation.....	23
9.2. Présentation du concept : la carte de visite AR interactive	23
9.3. Architecture technique de la solution	23
10. Préparation des assets et conception de la carte	23
10.1. Constitution d'une bibliothèque d'assets	23
10.2. Conception de la carte de visite.....	24
11. Création de la database Vuforia personnalisée	25
12. Mise en place de la scène Unity.....	27

12.1. Création d'une nouvelle scène.....	27
12.2. Configuration de l'AR Camera avec la nouvelle licence	27
12.3. Import de la database et création de l'Image Target.....	27
12.4. Premier test rapide	28
13. Intégration de l'avatar 3D animé	29
13.1. Préparation des fichiers Mixamo	29
13.2. Configuration du Rig en Humanoid.....	29
13.3. Création de l'Animator Controller.....	29
13.4. Placement de l'avatar dans la scène	31
13.5. Activation de la lecture en boucle	31
14. Ajout du texte 3D et des particules d'apparition	32
14.1. Texte 3D avec TextMeshPro	32
14.2. Système de particules d'apparition	32
14.3. Script de déclenchement à la détection	33
15. Boutons UI cliquables avec liens externes.....	33
15.1. Création du Canvas World Space	33
15.2. Création des boutons LinkedIn, GitHub et Email.....	34
15.3. Script d'ouverture d'URL.....	34
15.4. Activation du raycast pour rendre les boutons cliquables en AR	35
16. Intégration de l'audio spatial 3D.....	36
16.1. Configuration de l'AudioSource	36
16.2. Synchronisation avec le tracking Vuforia.....	36
17. Virtual Button : un easter egg interactif	37
17.1. Création du Virtual Button.....	37
17.2. Script de gestion du Virtual Button.....	37
18. Build APK et tests sur smartphone.....	39
18.1. Configuration du build	39
18.2. Génération et installation de l'APK.....	39
18.3. Tests fonctionnels	Erreur ! Signet non défini.
19. Difficultés rencontrées et solutions apportées.....	Erreur ! Signet non défini.
19.1. Boutons UI ne réagissant pas au clic.....	Erreur ! Signet non défini.
19.2. Animations qui ne s'enchaînent pas correctement	Erreur ! Signet non défini.
19.3. Build Android qui plante au démarrage.....	Erreur ! Signet non défini.
19.4. Tracking instable sur la première version de la carte	Erreur ! Signet non défini.
19.5. Audio trop fort ou pas spatialisé.....	Erreur ! Signet non défini.
20. Analyse critique et perspectives	Erreur ! Signet non défini.
20.1. Limitations de la version gratuite de Vuforia.....	Erreur ! Signet non défini.
20.2. Comparaison avec les alternatives modernes	Erreur ! Signet non défini.
20.3. Perspectives d'évolution du projet	Erreur ! Signet non défini.
20.4. Compétences acquises.....	Erreur ! Signet non défini.

21. Conclusion générale.....	Erreur ! Signet non défini.
Annexes.....	Erreur ! Signet non défini.
Annexe A — Code source des scripts C#.....	Erreur ! Signet non défini.
A.1. OpenURL.cs.....	Erreur ! Signet non défini.
A.2. AudioOnTracking.cs	Erreur ! Signet non défini.
A.3. ParticleOnDetection.cs	Erreur ! Signet non défini.
A.4. DanceButtonHandler.cs	Erreur ! Signet non défini.
Annexe B — Webographie et ressources utilisées	Erreur ! Signet non défini.
Annexe C — Récapitulatif des fonctionnalités explorées	Erreur ! Signet non défini.

1. Introduction

1.1. Contexte de l'atelier

Cet atelier s'inscrit dans le cadre de la découverte des technologies de Réalité Augmentée (RA, ou AR pour Augmented Reality en anglais). La réalité augmentée est une technologie qui consiste à superposer du contenu virtuel — images, modèles 3D, sons, textes — sur la perception du monde réel, généralement à travers l'écran d'un smartphone, d'une tablette ou d'un casque dédié.

Contrairement à la réalité virtuelle qui plonge l'utilisateur dans un environnement entièrement fictif, la RA enrichit le monde existant. Elle a aujourd'hui de nombreuses applications concrètes : marketing interactif, formation industrielle, éducation, jeux vidéo (Pokémon Go), maintenance assistée, médecine, design d'intérieur (IKEA Place), et bien d'autres.

1.2. Présentation de Vuforia Engine

Vuforia est l'un des moteurs de réalité augmentée les plus utilisés au monde. Il a été développé initialement par Qualcomm puis racheté par PTC. Il s'intègre nativement dans le moteur de jeu Unity, ce qui en fait une solution privilégiée pour le développement rapide d'applications AR multi-plateformes (Android, iOS, lunettes connectées).

Le principe de fonctionnement repose sur la vision par ordinateur : la caméra de l'appareil capte un flux vidéo, Vuforia analyse en temps réel chaque image pour détecter des cibles préalablement enregistrées (images, objets 3D, codes), puis Unity superpose le contenu virtuel à la position et l'orientation correctes.

Pourquoi Vuforia plutôt qu'une autre solution ?

- Compatible avec une large gamme d'appareils, y compris ceux qui ne supportent pas ARCore ou ARKit.
- Marker-based tracking robuste : la reconnaissance d'images planes est plus stable qu'avec ARCore seul.
- Version gratuite (Basic) suffisante pour le prototypage et l'apprentissage.
- Documentation et communauté abondantes.

1.3. Objectifs du compte rendu

Ce document poursuit deux objectifs distincts :

1. Présenter de manière détaillée et pédagogique la réalisation du TP de base tel qu'il a été demandé par les encadrants : installation de l'environnement, configuration de la scène AR, intégration d'un personnage animé sur une cible image, et build Android.
2. Démontrer une démarche d'approfondissement personnelle en explorant des fonctionnalités avancées de Vuforia non couvertes par le TP de base, et en réalisant un projet d'innovation concret : une « Carte de visite AR interactive ». Cette deuxième partie a pour but de prouver une réelle compréhension de la technologie, au-delà de la simple exécution d'instructions.

PARTIE I — Réalisation du TP de base

Cette première partie présente, étape par étape, la réalisation du TP de base tel qu'il a été demandé. Chaque manipulation est accompagnée d'une explication de son utilité afin de comprendre non seulement « comment » faire mais aussi « pourquoi » le faire.

2. Préparation de l'environnement de travail

2.1. Vérification des prérequis

Avant de commencer le développement, il est essentiel de s'assurer que tous les outils nécessaires sont installés et configurés. Cette étape, souvent négligée, évite de nombreux problèmes en cours de réalisation.

Les prérequis pour ce TP sont :

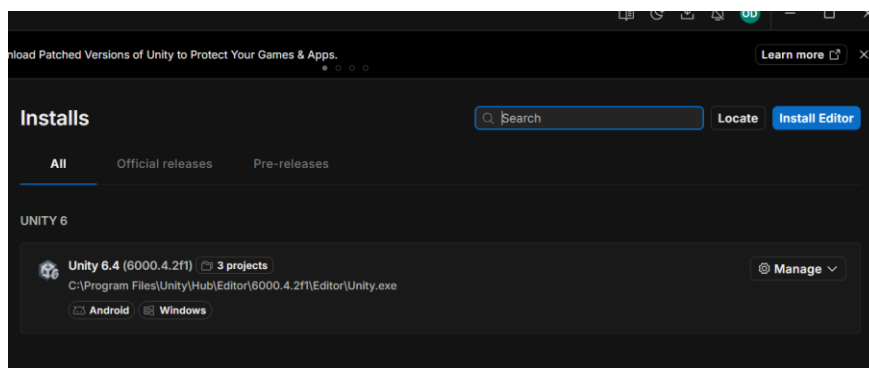
- Unity Hub installé sur la machine, avec une version d'Unity compatible avec Vuforia. Le sujet recommande Unity 2021.3 LTS, mais les versions plus récentes sont également compatibles.
- Un compte Unity ID actif, indispensable pour l'activation de l'éditeur.
- Un compte Vuforia Developer (gratuit), créé sur le portail officiel.
- Une connexion Internet stable pour télécharger les packages.

Choix de la version Unity :

Pour la réalisation de ce TP, j'ai utilisé Unity 6.4 (build 6000.4.2f1), une version plus récente que la 2021.3 LTS suggérée par le sujet. Ce choix s'explique par le fait que cette version était déjà installée sur ma machine pour des projets antérieurs et par manque d'espace sur ma machine je n'ai pas installé la version 2021. Le sujet du TP précise lui-même que les versions plus récentes restent compatibles avec Vuforia, et j'ai pu vérifier que le TP fonctionne parfaitement avec cette version.

Note sur la compatibilité des versions Unity

Le SDK Vuforia est régulièrement mis à jour pour rester compatible avec les versions récentes d'Unity. À ce jour, les versions 2021.3 LTS, 2022.x, 2023.x et Unity 6 sont toutes supportées. L'interface peut présenter de légères différences entre versions (positions de menus, libellés), mais les fonctionnalités principales restent identiques. Les captures d'écran de ce rapport correspondent à l'interface d'Unity 6.4.



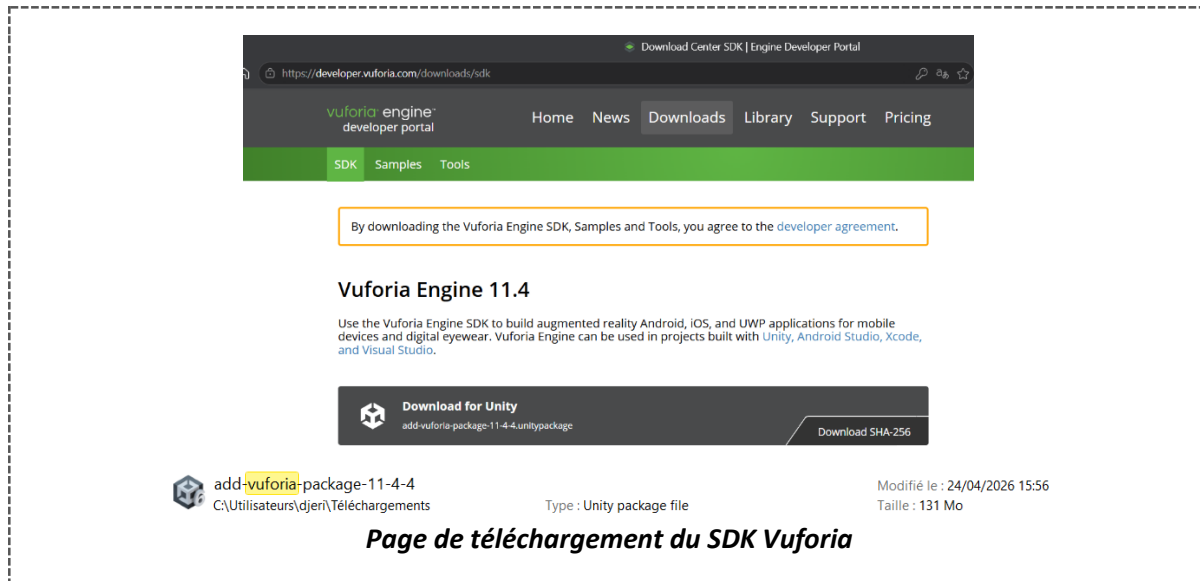
Unity Hub avec la version Unity 6.4 (6000.4.2f1) installée

2.2. Téléchargement du package Vuforia Engine

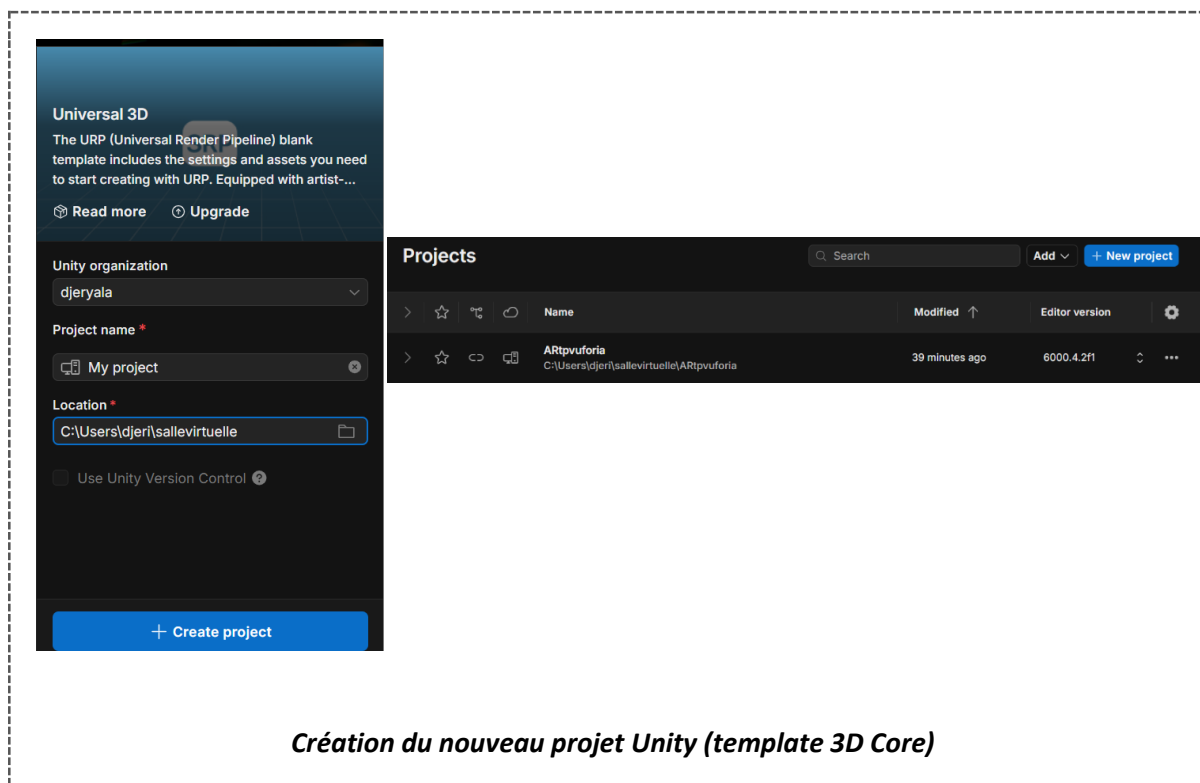
Vuforia n'est pas inclus par défaut dans Unity. Il faut donc le télécharger séparément et l'importer dans le projet. Le package est fourni au format .unitypackage, un format propriétaire d'Unity qui permet de regrouper plusieurs assets, scripts et configurations en un seul fichier.

Procédure suivie :

3. Connexion au portail Vuforia : <https://developer.vuforia.com/downloads/sdk>
4. Téléchargement du fichier Vuforia Engine Package for Unity (.unitypackage).
5. Sauvegarde du fichier dans un dossier facile à retrouver.

**2.3. Création du projet Unity et import du package**

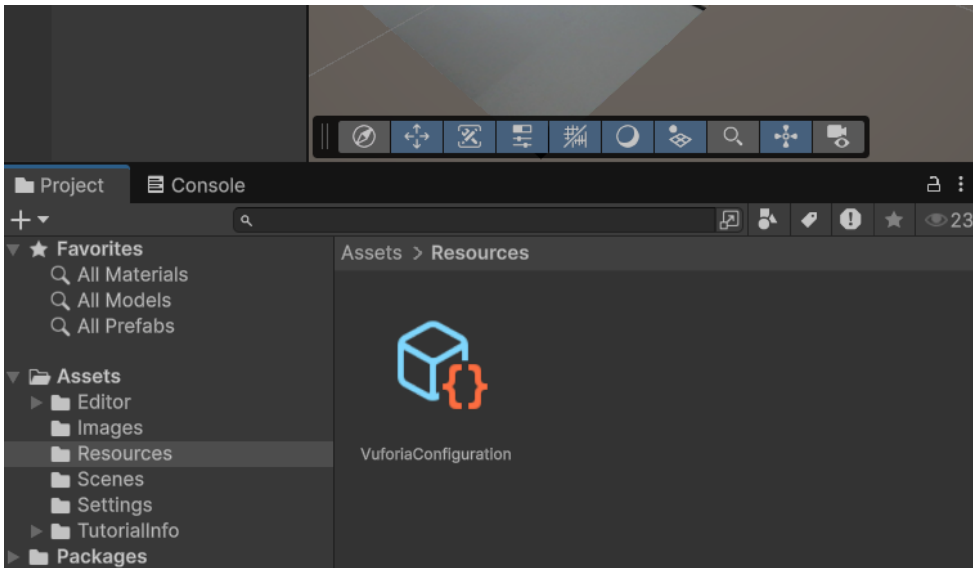
Une fois Unity Hub ouvert, j'ai créé un nouveau projet 3D (Core 3D) que j'ai nommé pour bien identifier le contexte du travail. Le choix du template 3D Core est important car il fournit la configuration de rendu standard adaptée à Vuforia.



Pour importer le package Vuforia dans le projet, deux méthodes équivalentes sont possibles :

- Méthode 1 — Menu Unity : Assets > Import Package > Custom Package, puis sélection du fichier .unitypackage.

- Méthode 2 — Glisser-déposer (recommandée) : faire un drag and drop direct du fichier .unitypackage dans la fenêtre Project d'Unity. Unity lance automatiquement l'assistant d'importation. Cette méthode est plus rapide et plus naturelle.

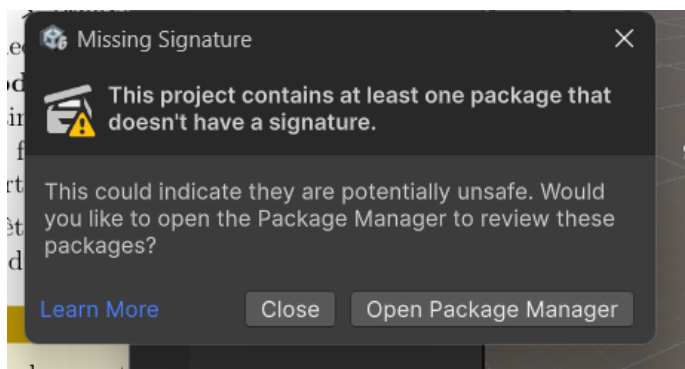
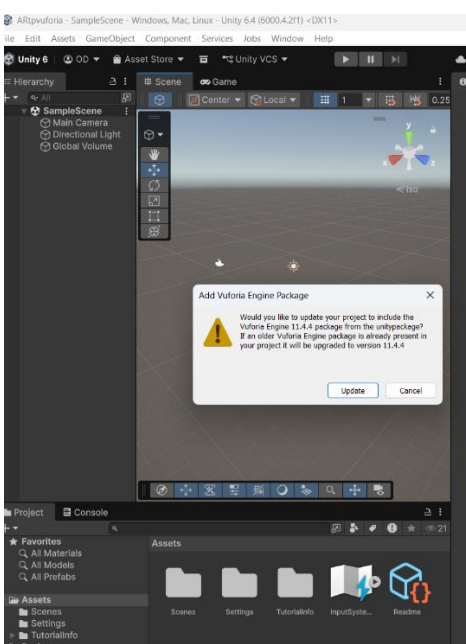


Fenêtre d'import Vuforia avec la liste des composants à importer

Dans la fenêtre d'import, j'ai cliqué sur Import All afin d'importer la totalité des composants du package, ce qui garantit que toutes les fonctionnalités seront disponibles.

Avertissements de dépréciation

Lors de l'import, Unity peut afficher des avertissements (warnings) jaunes dans la console concernant des fonctions dépréciées. Ces messages sont sans conséquence pour notre TP : ils signalent simplement que certaines anciennes fonctions seront supprimées dans des versions futures. Seules les erreurs (en rouge) doivent réellement nous inquiéter.



Console Unity après import — aucune erreur rouge

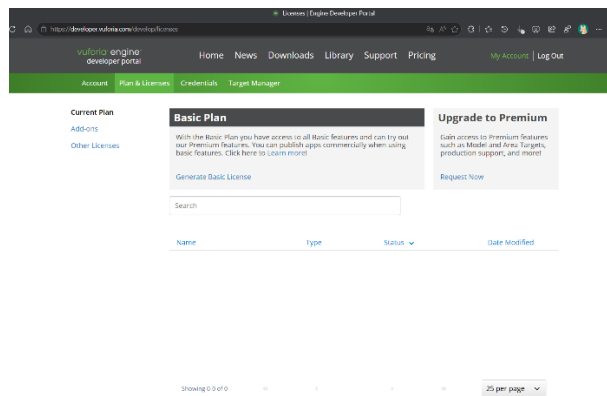
3. Configuration de la licence Vuforia

3.1. Génération de la clé de licence

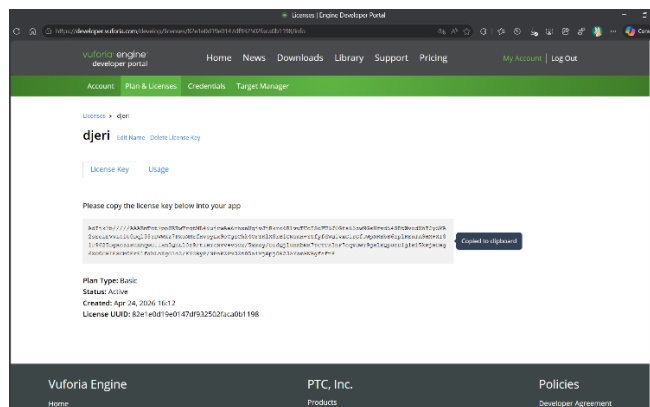
Pour des raisons de suivi et de monétisation, Vuforia nécessite une clé de licence pour chaque application développée. La version Basic est entièrement gratuite et largement suffisante pour notre TP.

Étapes suivies :

6. Connexion au portail Vuforia : <https://developer.vuforia.com>
7. Navigation vers le tableau de bord, puis License Manager > Add License Key.
8. Sélection du type Basic (gratuit) et attribution d'un nom à l'application.
9. Validation par Confirm puis copie de la clé générée.



Page License Manager — création d'une licence Basic



Clé de licence générée (à copier)

3.2. Insertion de la clé dans Unity

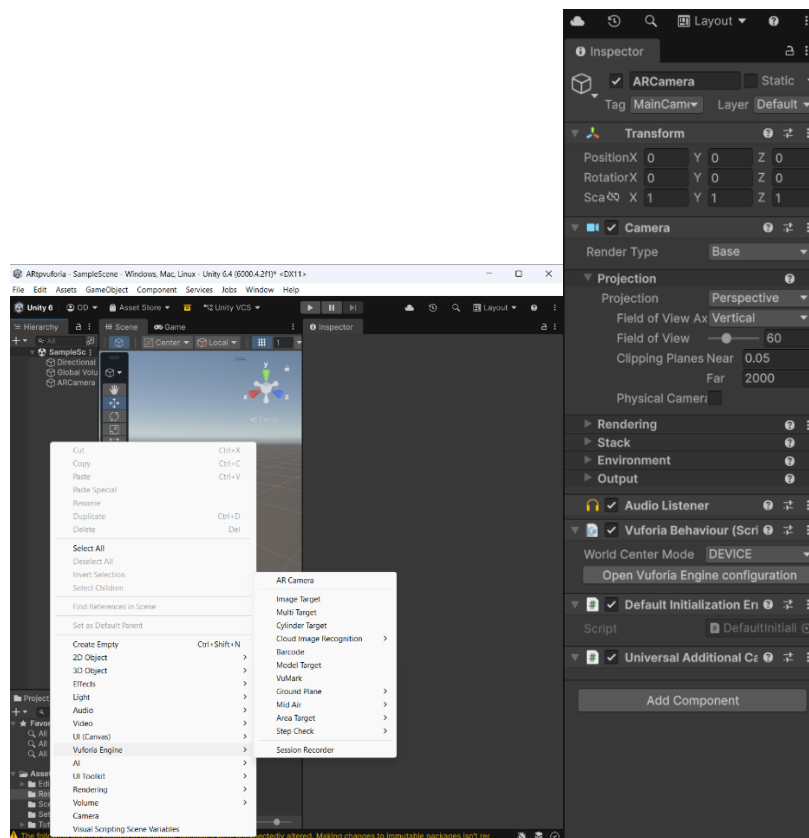
La clé doit ensuite être collée dans la configuration Vuforia du projet Unity. Il est important de noter que cette configuration se fait dans l'Inspector via l'AR Camera, et non dans Project Settings.

Procédure :

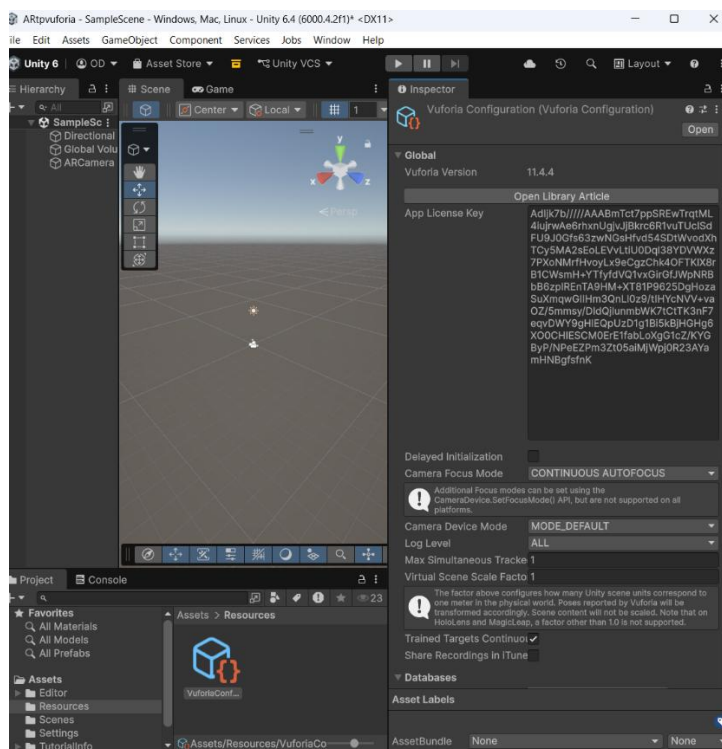
10. Création d'une scène contenant une AR Camera (voir section suivante) ou utilisation de la scène existante.
11. Sélection de l'objet AR Camera dans la fenêtre Hierarchy.
12. Dans l'Inspector, repérage du composant Vuforia Behaviour (Script).

13. Clic sur le bouton Open Vuforia Configuration.

14. Dans le champ App License Key, collage de la clé précédemment copiée (sans espaces parasites en début ou fin).



Inspector de l'AR Camera avec le bouton Open Vuforia Configuration



Champ App License Key rempli avec la clé

Erreur fréquente

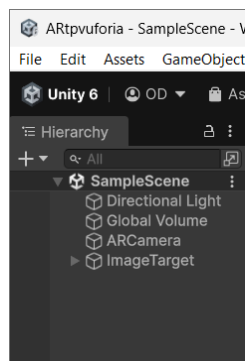
Si la clé est collée avec des espaces ou des caractères invisibles, Vuforia ne se lance pas et l'écran reste noir au démarrage. Il faut alors recopier la clé proprement depuis le portail.

4. Configuration de la scène AR**4.1. Suppression de la Main Camera et ajout de l'AR Camera**

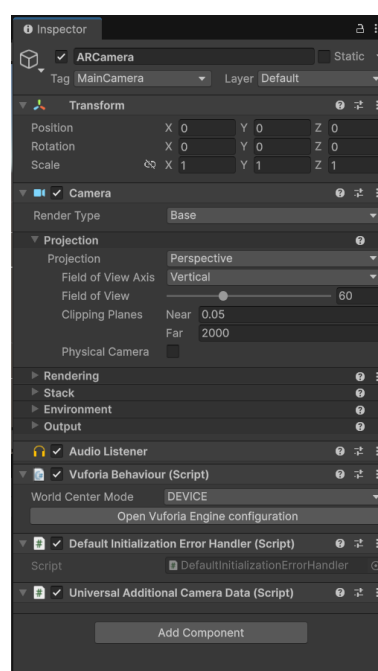
Toute scène Unity contient par défaut une Main Camera. Cependant, Vuforia utilise sa propre caméra spécialisée — l'AR Camera — qui inclut tous les composants nécessaires au tracking en temps réel. Garder les deux caméras simultanément provoquerait des conflits de rendu et un comportement instable.

Procédure :

15. Sélection de Main Camera dans la Hierarchy.
16. Suppression avec la touche Suppr (ou Backspace).
17. Clic droit dans la Hierarchy > Vuforia Engine > AR Camera.
18. Vérification : un nouvel objet AR Camera apparaît dans la hiérarchie avec ses composants Vuforia.



Hierarchy après suppression de Main Camera et ajout de AR Camera



Inspector de l'AR Camera montrant le composant Vuforia Behaviour

4.2. Ajout de l'Image Target

L'Image Target est l'objet virtuel qui représente l'image physique que la caméra devra reconnaître. C'est sur cet objet que viendra se positionner notre contenu 3D (le personnage, l'animation, etc.).

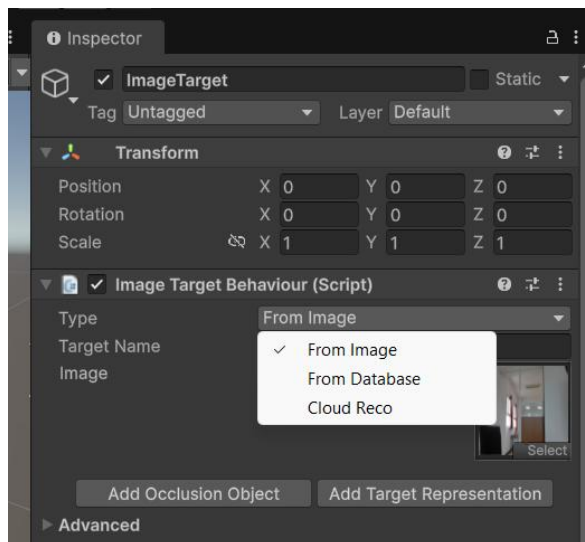
Procédure :

19. Clic droit dans la Hierarchy > Vuforia Engine > Image Target.
20. Sélection de l'objet Image Target créé.
21. Dans l'Inspector, dans le composant Image Target Behaviour, choix du type de source de l'image.

Deux options sont disponibles :

- From Image : import direct d'une image depuis les Assets du projet (PNG ou JPG, riche en détails).
- From Database : utilisation d'une base de données créée sur le portail Vuforia (méthode plus avancée, utilisée dans la Partie II de ce rapport).

Pour le TP de base, j'ai utilisé la méthode From Image avec une image préparée à l'avance, riche en détails contrastés afin d'optimiser la qualité du tracking.



Inspector de l'Image Target avec le type From Image sélectionné

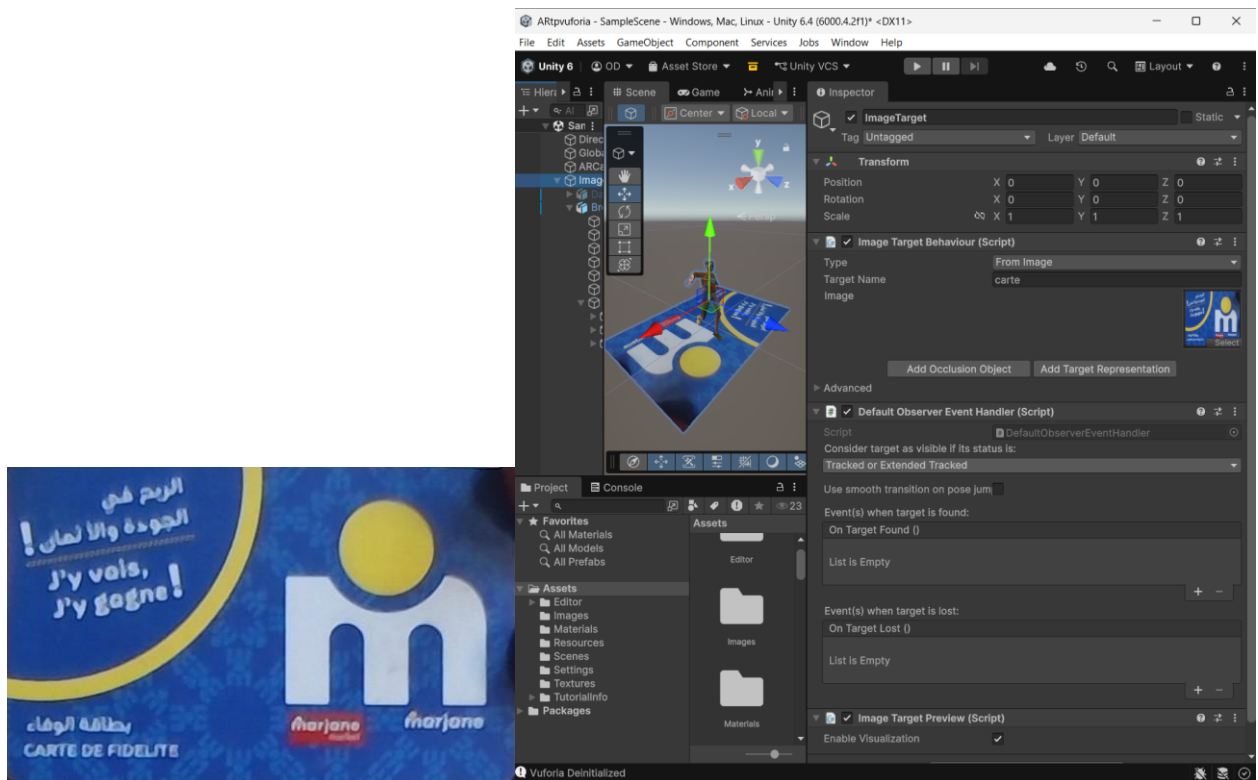


Image cible utilisée pour le TP de base

L'Image Target a été positionnée au centre de la scène avec les coordonnées (0, 0, 0) afin de servir de point de référence pour les éléments enfants.

L'Inspector de l'Image Target propose deux fonctionnalités avancées non exploitées dans le cadre de ce projet mais pertinentes à mentionner. **Add Target Representation** permet d'afficher un maillage de prévisualisation de la cible dans la Scene view, particulièrement utile pour les Model Targets 3D complexes. **Add Occlusion Object** permet d'ajouter un objet invisible qui masque le contenu 3D situé derrière lui, créant ainsi un effet d'occlusion réaliste lorsque le contenu virtuel doit interagir physiquement avec des objets du monde réel. Ces deux fonctionnalités prendraient tout leur sens dans une évolution future du projet où l'avatar interagirait avec un environnement physique plus complexe qu'une simple carte plate.

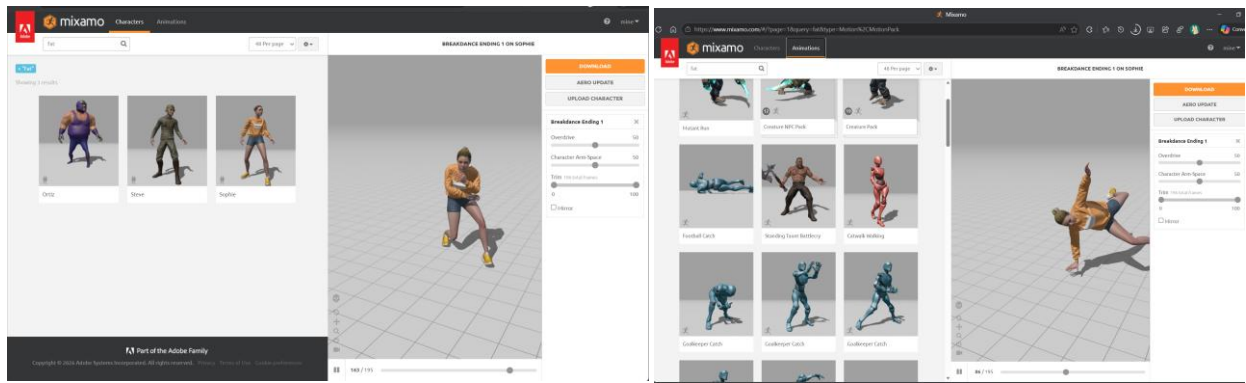
5. Intégration du contenu 3D animé

5.1. Récupération d'un personnage et d'une animation sur Mixamo

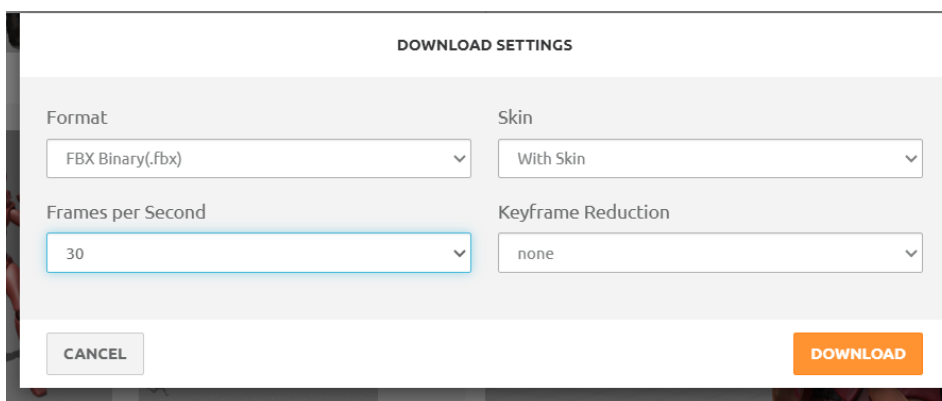
Mixamo est un service en ligne gratuit proposé par Adobe qui met à disposition des centaines de personnages et d'animations 3D prêts à l'emploi. Cela évite de devoir modéliser et animer soi-même un personnage, ce qui est un travail extrêmement long.

Procédure :

22. Connexion sur <https://www.mixamo.com> avec un compte Adobe.
23. Onglet Characters : sélection d'un personnage (par exemple Y Bot, ou un personnage humanoïde plus stylisé).
24. Onglet Animations : recherche et sélection d'une animation (par exemple Idle, Dance, Wave).
25. Téléchargement avec les options : Format = FBX for Unity (.fbx), Skin = With Skin (pour inclure le maillage et les textures), Frames per Second = 30.



Site Mixamo — sélection du personnage et choix d'une animation



Site Mixamo — options de téléchargement

Justification des choix de paramètres de téléchargement

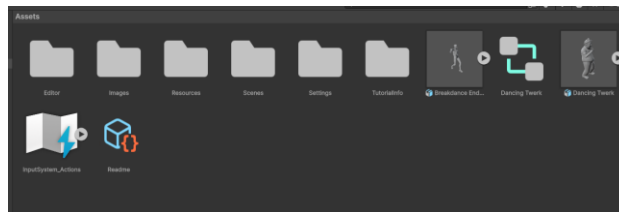
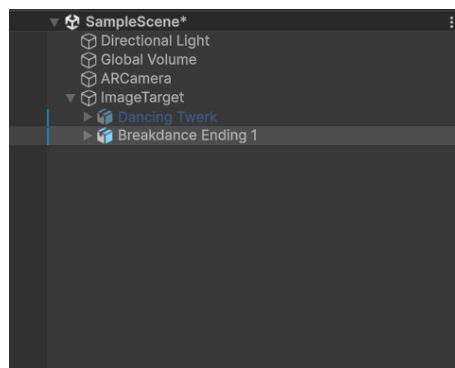
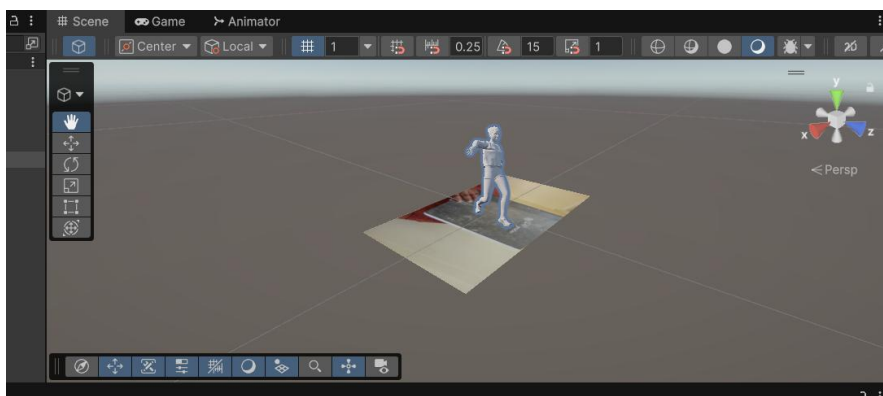
Le choix des paramètres de téléchargement n'est pas anodin et résulte d'une réflexion sur l'optimisation du pipeline d'import vers Unity. Le format **FBX Binary** est privilégié pour sa compacité et sa rapidité de chargement par rapport au format ASCII équivalent — il est trois à cinq fois plus léger et offre la meilleure compatibilité avec Unity, qui le considère comme son format standard pour l'animation humanoïde. L'option **With Skin** est sélectionnée uniquement lors du premier téléchargement afin de récupérer le maillage visible et les textures du personnage ; les animations supplémentaires (par exemple Dance et Greeting) sont quant à elles téléchargées en Without Skin, ce qui permet de mutualiser un seul mesh pour plusieurs animations via l'Animator Controller, économisant ainsi à la fois l'espace de stockage et la mémoire vive à l'exécution. Le réglage de **30 frames per second** correspond au standard de l'animation mobile : il offre une fluidité visuelle suffisante à l'œil humain tout en limitant la consommation de ressources, un point essentiel dans un contexte AR où le matériel est déjà fortement sollicité par le tracking Vuforia, le rendu 3D et le flux caméra simultanés. Enfin, l'option **Keyframe Reduction = None** garantit que l'animation est récupérée dans sa qualité brute sans compression destructive ; le gain d'espace serait de toute manière négligeable au regard de la taille totale du projet, et il est préférable de laisser Unity gérer lui-même la compression d'animation en aval, via les options dédiées de l'Inspector, plutôt que de la subir dès la source.

5.2. Intégration du personnage dans Unity

Le personnage téléchargé doit ensuite être intégré dans la scène Unity de telle sorte qu'il soit affiché lorsque l'image cible est détectée.

Étapes :

26. Glissement du fichier .fbx téléchargé dans le dossier Assets du projet Unity.
27. Glissement du personnage depuis Assets vers la Hierarchy, en le déposant directement sur l'objet Image Target. Le personnage devient ainsi un enfant de l'Image Target, ce qui signifie qu'il suivra automatiquement la position et l'orientation de la cible.
28. Ajustement de la Position et de la Scale du personnage pour qu'il soit proportionnel à l'image. Avec une image cible de quelques centimètres, une Scale autour de 0.05 à 0.1 est généralement appropriée.

**Personnage importé dans Assets****Hierarchy avec le personnage en enfant de l'Image Target****Vue Scene montrant le personnage positionné sur l'Image Target****Pourquoi le personnage doit-il être enfant de l'Image Target ?**

Dans Unity, lorsqu'un objet est enfant d'un autre dans la Hierarchy, il hérite de ses transformations (position, rotation, échelle). Ainsi, lorsque Vuforia détecte la cible et met à jour sa position en temps réel, le personnage suit automatiquement, sans aucun script. C'est élégant et performant.

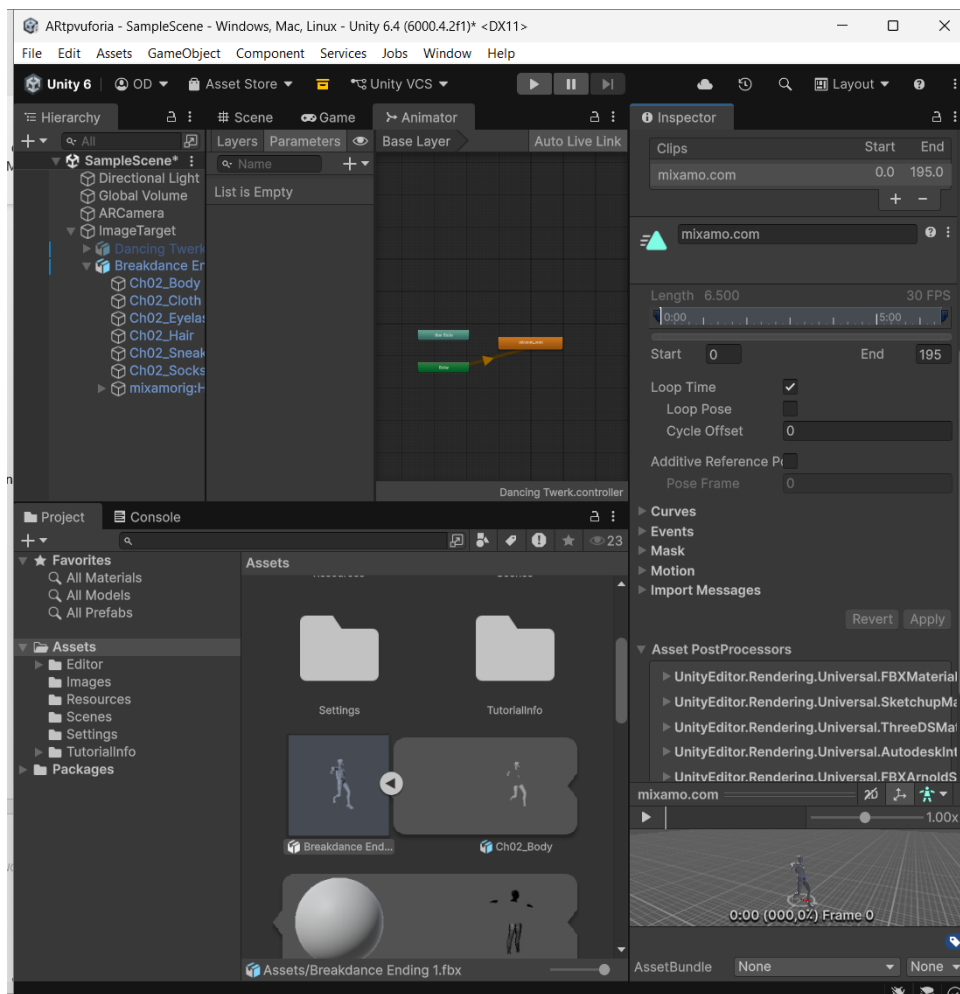
5.3. Configuration de l'animation en boucle

Par défaut, une animation Mixamo s'exécute une seule fois puis s'arrête. Pour qu'elle tourne continuellement tant que la cible est visible, il faut activer l'option Loop Time.

Pour qu'un personnage puisse jouer une animation dans Unity, il ne suffit pas d'importer un fichier FBX. Unity nécessite la mise en place d'un objet intermédiaire appelé Animator Controller, qui agit comme un chef d'orchestre décidant quelle animation jouer et comment passer d'une animation à l'autre. Sans cet objet, le personnage reste figé en position T-pose (bras écartés en croix), ce qui est l'erreur la plus fréquente lors d'un premier projet d'animation Unity.

Procédure :

29. Ouverture des fenêtres d'animation : Window > Animation > Animator.
30. Sélection du clip d'animation du personnage dans la fenêtre Animation.
31. Dans l'Inspector du clip, activation de la case Loop Time.

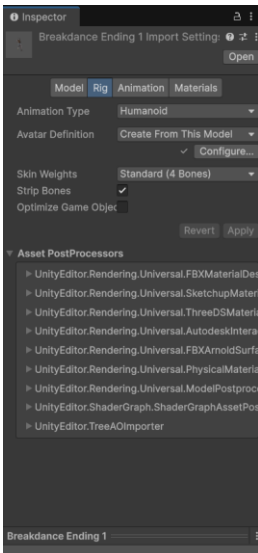


Fenêtre Animator du personnage et Option Loop Time cochée dans l'Inspector du clip

5.3.1. Configuration du Rig en mode Humanoid

Avant tout, le rig (squelette) du personnage doit être configuré en mode Humanoid. Cette configuration permet à Unity de comprendre que le modèle est un personnage humain à deux bras et deux jambes, et autorise la réutilisation d'animations entre différents personnages humanoïdes Mixamo.

Procédure : sélection du FBX dans Assets, onglet **Rig** dans l'Inspector, Animation Type = Humanoid, Avatar Definition = Create From This Model, puis Apply.



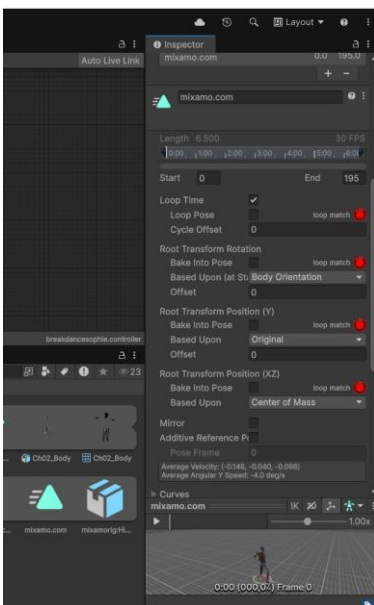
Onglet Rig avec Animation Type Humanoid

5.3.2. Extraction et configuration du clip d'animation

Le fichier FBX importé contient à la fois le mesh du personnage et le clip d'animation correspondant. Pour accéder au clip, il faut déplier le FBX dans la fenêtre Project en cliquant sur la petite flèche à sa gauche. Le clip apparaît avec une icône en triangle bleu.

Une fois le clip sélectionné, plusieurs options sont configurées dans son Inspector :

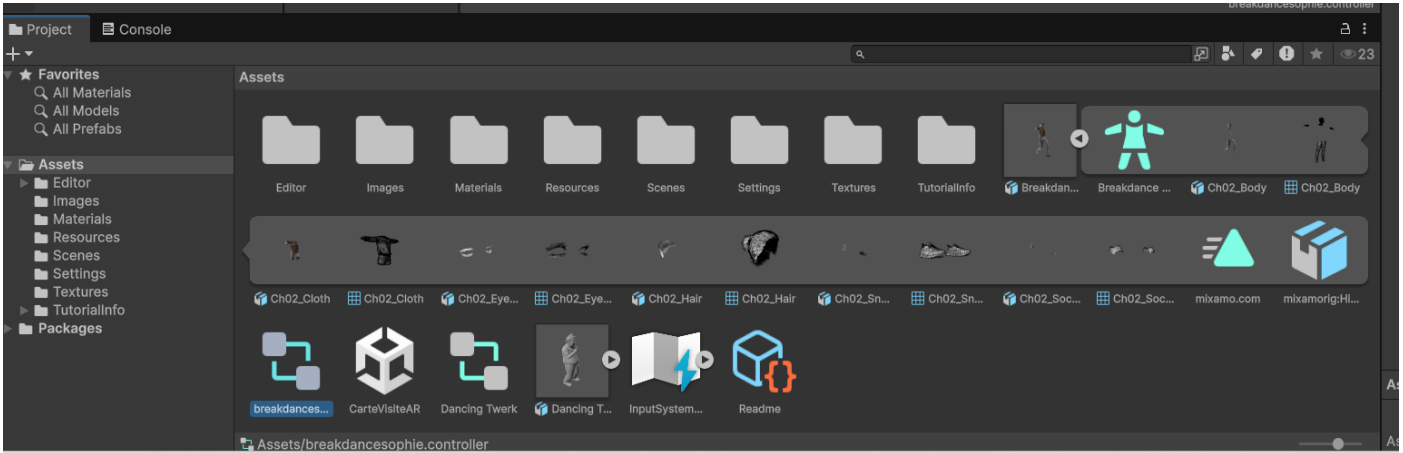
- Renommage du clip avec un nom parlant (par exemple Dance plutôt que le mixamo.com par défaut).
- Activation de **Loop Time** pour que l'animation tourne en boucle plutôt que de s'exécuter une seule fois.
- Activation de **Loop Pose** pour assurer une transition fluide entre la dernière et la première frame, évitant les saccades à chaque cycle.



Inspector du clip avec Loop Time et Loop Pose cochés

5.3.3. Création de l'Animator Controller

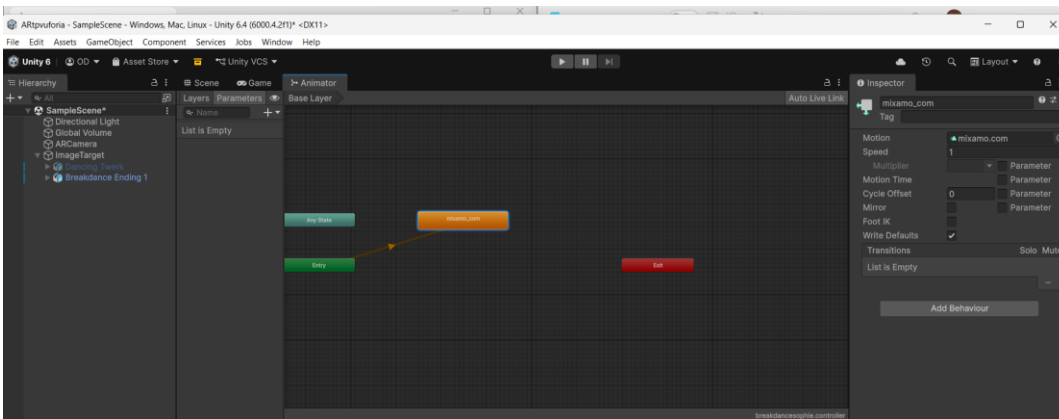
L'Animator Controller est créé via un clic droit dans le dossier Assets > Create > Animator Controller, puis nommé CharacterAnimator. Un nouveau fichier au format .controller apparaît dans le dossier.



Nouveau fichier Animator Controller créé

5.3.4. Ajout du clip dans l'Animator

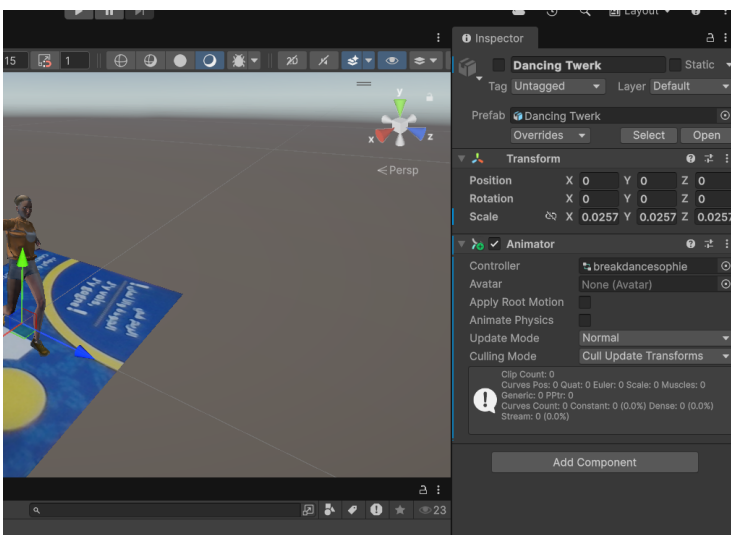
Un double-clic sur le fichier CharacterAnimator ouvre la fenêtre Animator, qui présente trois nœuds par défaut : Entry, Any State et Exit. Le clip d'animation est alors glissé-déposé directement depuis le dossier Assets dans cette fenêtre. Unity crée automatiquement un nouvel état orange portant le nom du clip et trace une flèche depuis Entry vers ce nouvel état, désignant ainsi cette animation comme état par défaut joué au démarrage de la scène.



Fenêtre Animator avec le clip comme état par défaut

5.3.5. Liaison de l'Animator Controller au personnage

La dernière étape consiste à connecter l'Animator Controller au GameObject du personnage présent dans la scène. Lorsque le FBX a été glissé dans la Hierarchy, Unity a automatiquement ajouté un composant Animator au personnage. Ce composant possède deux champs importants : Controller (initialement vide) et Avatar (rempli automatiquement par Unity à partir du Rig). Le fichier CharacterAnimator créé précédemment est glissé-déposé dans le champ Controller, ce qui active la connexion entre l'orchestrateur et le personnage.



Inspector du personnage avec composant Animator et Controller assigné

5.3.6. Validation du fonctionnement

Le test final s'effectue en mode Play dans Unity. Lorsque la cible image est présentée à la webcam, le personnage doit apparaître et jouer son animation en boucle de manière fluide. Si le personnage reste figé en T-pose, cela signifie qu'une des étapes précédentes a été oubliée — le plus souvent le champ Controller du composant Animator qui est resté vide.

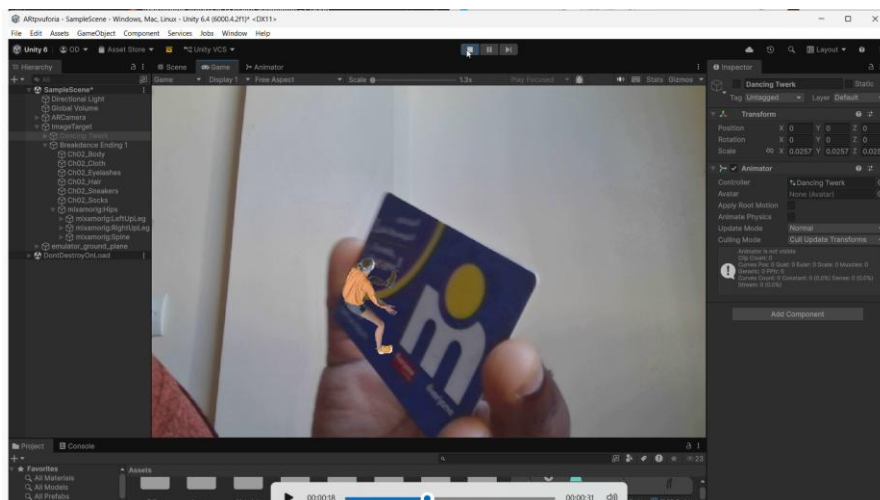
6. Phase de test dans l'éditeur Unity

6.1. Test avec la webcam du laptop

Avant de procéder au build Android, il est essentiel de valider que la scène fonctionne correctement dans l'éditeur. Unity permet de tester directement avec la webcam du laptop, ce qui évite des cycles de build inutiles.

Procédure :

32. Vérification que la webcam est connectée et reconnue par le système d'exploitation.
33. Clic sur le bouton Play (triangle) en haut de l'interface Unity.
34. La fenêtre Game affiche le flux vidéo de la webcam.
35. Présentation de l'image cible imprimée (ou affichée sur un écran secondaire) devant la webcam.
36. Si la configuration est correcte, le personnage 3D apparaît sur l'image et l'animation tourne en boucle.



Enregistrement
2026-05-01 175807.m

Fenêtre Game affichant le flux webcam et Personnage 3D apparaissant sur l'image cible (test webcam)

Critères de validation respectés :

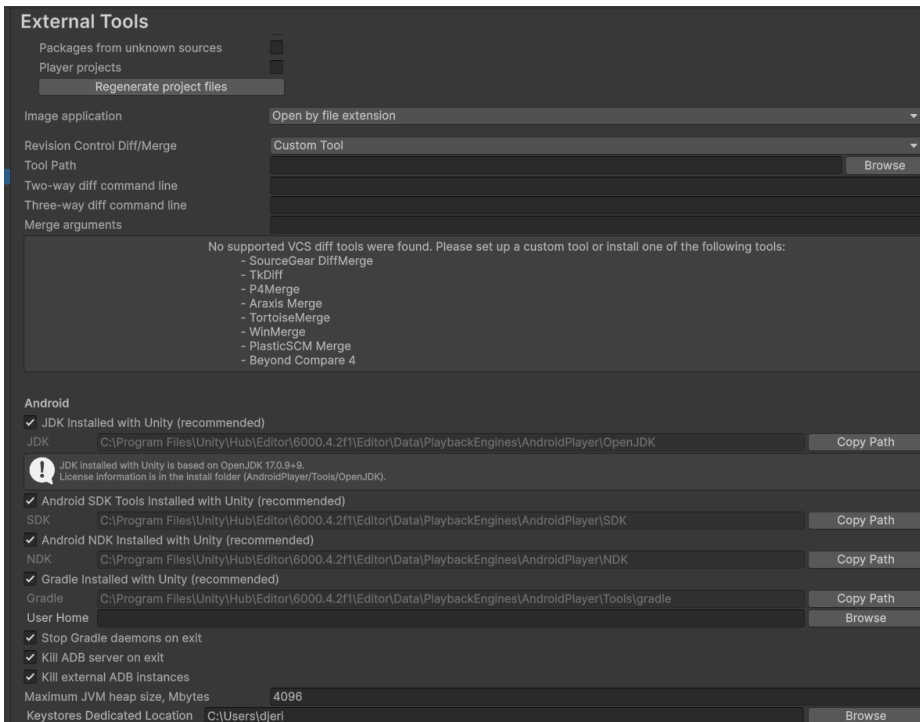
- Le flux webcam s'affiche correctement dans la fenêtre Game.
- Lorsque l'image cible est présentée à la caméra, le personnage 3D apparaît dessus.
- L'animation joue en boucle de manière fluide.
- Lorsque l'image est cachée, le personnage disparaît instantanément.

7. Création du Build Android (APK)

7.1. Prérequis pour le build Android

Pour générer un APK fonctionnel, plusieurs composants doivent être présents :

- Le module Android Build Support installé via Unity Hub (avec OpenJDK et Android SDK).
- Les chemins SDK et JDK correctement configurés dans Edit > Preferences > External Tools.
- Un smartphone Android avec le mode développeur activé pour les tests par USB.



Modules installés dans Unity Hub

7.2. Paramétrage du projet pour Android

Procédure :

37. Ouverture de File > Build Settings.
38. Sélection d'Android dans la liste des plateformes.
39. Clic sur Switch Platform et attente de la recompilation des assets.
40. Ouverture de Player Settings et configuration : Company Name, Product Name, et activation de OpenGL ES3 dans Other Settings > Rendering.

Fenêtre Build Settings avec Android sélectionné

Player Settings — section Rendering avec OpenGL ES3 activé

7.3. Configuration cruciale du XR Plug-in Management

Étape critique

Avant de lancer le build Android, il est impératif de désactiver tous les plug-in providers (ARCore, OpenXR, etc.) dans XR Plug-in Management pour Android. Sans cela, des conflits surviennent avec

Vuforia et l'application plante au démarrage. C'est une erreur très fréquente qui peut faire perdre des heures de débogage.

Procédure :

41. Edit > Project Settings > XR Plug-in Management.
42. Sélection de l'onglet Android (icône robot vert).
43. Décochage de TOUS les Plug-in Providers présents dans la liste.
44. Vérification que la liste des providers cochés est totalement vide.

XR Plug-in Management — Android — tous les providers décochés

7.4. Lancement du build et installation sur smartphone

Procédure :

45. Retour à File > Build Settings.
46. Clic sur Add Open Scenes pour ajouter la scène courante.
47. Vérification que la scène apparaît dans la liste Scenes In Build.
48. Clic sur Build (ou Build And Run si le téléphone est connecté).
49. Choix de l'emplacement de sauvegarde du fichier APK et lancement de la compilation.

Build en cours (barre de progression)

Fichier APK généré dans le dossier de sortie

Pour installer l'APK sur un téléphone Android sans passer par le Play Store :

- Activer l'installation depuis des sources inconnues dans les paramètres de sécurité du téléphone.
- Transférer le fichier .apk via USB ou par email.
- Lancer le fichier sur le téléphone et accepter l'installation.

APK installé sur le smartphone

Application AR en fonctionnement sur smartphone

8. Conclusion de la partie 1

Le TP de base a été réalisé avec succès. Toutes les étapes demandées par les encadrants ont été suivies : préparation de l'environnement, configuration de la licence Vuforia, mise en place de la scène AR avec une caméra et une cible image, intégration d'un personnage Mixamo animé en boucle, et enfin génération d'un APK fonctionnel testé sur smartphone.

Cependant, comme mentionné en introduction, j'ai souhaité aller plus loin que le strict cadre du TP. Les encadrants ont en effet encouragé les étudiants à pousser le travail, à découvrir en profondeur les fonctionnalités de Vuforia, et à imaginer des cas d'usage utiles. C'est l'objet de la deuxième partie de ce rapport.

PARTIE II — Travail d'innovation et d'approfondissement

9. Démarche et choix du projet d'innovation

9.1. Motivation

Le TP de base permet de découvrir les bases de Vuforia et d'obtenir un résultat visuellement satisfaisant. Cependant, il n'exploite qu'une infime partie des capacités du moteur. De nombreuses fonctionnalités avancées restent inexplorées : reconnaissance de plusieurs cibles simultanément, boutons virtuels physiquement intégrés à la cible, audio spatialisé, particules, interaction tactile, liens vers le monde réel.

De plus, un projet AR n'a de sens que s'il répond à un besoin concret. J'ai donc choisi de concevoir une application réutilisable au-delà du cadre académique : une carte de visite étudiante en réalité augmentée.

9.2. Présentation du concept : la carte de visite AR interactive

Le principe est simple en apparence mais riche techniquement : une carte de visite imprimée standard, lorsqu'elle est scannée par l'application AR, devient un mini-CV interactif animé. Au-dessus de la carte apparaît un avatar 3D qui salue, accompagné d'un texte 3D présentant le nom et la filière, d'une musique d'ambiance, d'effets de particules à l'apparition, et de boutons cliquables qui ouvrent directement le profil LinkedIn, le compte GitHub ou la messagerie email de l'étudiant.

Ce concept présente plusieurs avantages :

- Il combine plusieurs fonctionnalités Vuforia et Unity dans un projet cohérent.
- Il a une véritable utilité dans la vie réelle — la carte peut être utilisée lors de salons, de rencontres professionnelles ou d'entretiens.
- Il est démontrable rapidement (moins d'une minute) tout en étant impressionnant.
- Il permet de manipuler des notions techniques variées : tracking, animation Humanoid, UI World Space, audio 3D, particules, événements Vuforia, ouverture d'URL externes.

9.3. Architecture technique de la solution

Le projet repose sur l'imbrication suivante d'objets dans la scène Unity :

- Une AR Camera Vuforia, racine de la scène, qui gère le tracking en temps réel.
- Un Image Target lié à une Database Vuforia personnalisée contenant l'image de la carte de visite.
- En enfant de l'Image Target : l'avatar 3D, un système de particules, un Canvas World Space contenant les boutons UI cliquables, et deux objets TextMeshPro pour le nom et la filière.
- Sur l'avatar : un Animator Controller gérant trois animations (Greeting, Idle, Dance) avec une transition automatique Greeting → Idle pour rendre le personnage vivant après son salut, et une AudioSource spatialisée diffusant la voix off.
- Plusieurs scripts C# personnalisés : OpenURL pour ouvrir les liens externes, AudioOnTracking pour synchroniser la voix off avec la détection de la cible, et ParticleOnDetection pour rejouer l'effet de particules à chaque réapparition.
- Un quatrième bouton UI dédié au déclenchement de l'animation Dance, créé en remplacement du Virtual Button Vuforia (déprécié dans les versions récentes du SDK).

10. Préparation des assets et conception de la carte

10.1. Constitution d'une bibliothèque d'assets

Pour préserver l'intégrité du travail réalisé en Partie I, j'ai choisi de développer le projet d'innovation dans une scène distincte (CarteVisiteAR.unity) au sein du même projet Unity. Cette organisation respecte les bonnes pratiques de

gestion de projet : isolation des fonctionnalités, possibilité de comparer les deux livrables, et génération de deux builds APK indépendants à partir d'une base d'environnement commune (Vuforia, licence, packages). La scène du TP de base reste ainsi intacte comme témoin du travail demandé, tandis que la scène d'innovation peut évoluer librement sans risque de régression.

Organisation du dossier :

- carte/ — image de la carte de visite au format PNG haute résolution.
- audio/ — fichier MP3 de musique d'ambiance libre de droits.
- personnage/ — fichiers FBX de Mixamo (personnage avec skin + animations sans skin).
- icones/ — icônes plates LinkedIn, GitHub, Email au format PNG transparent.

10.2. Conception de la carte de visite

La conception de l'image cible est l'étape la plus importante du projet : la qualité du tracking dépend directement de la richesse visuelle de l'image. Vuforia attribue un rating en étoiles (de 1 à 5) à chaque image téléversée sur son portail, basé sur le nombre et la distribution des feature points (points caractéristiques) détectés.

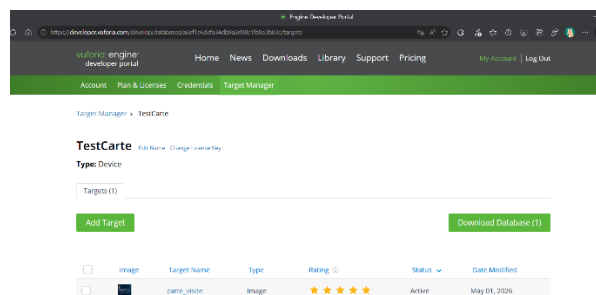
Règles de conception pour un rating 5 étoiles :

- Beaucoup de détails visuels — textures, motifs, photos.
- Contraste élevé entre les zones.
- Asymétrie générale — pas de design parfaitement centré ou symétrique.
- Distribution homogène des éléments — pas tout concentré dans une zone.
- Éviter les fonds unis, les dégradés simples et les patterns répétitifs.

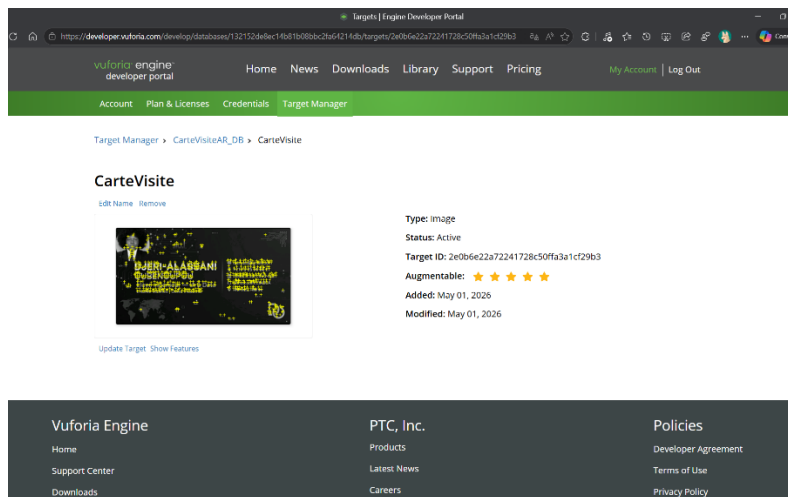
J'ai conçu la carte sur Canva en intégrant : une photo de profil en arrière-plan légèrement assombrie pour la lisibilité, mon nom en grandes lettres, ma filière, des coordonnées (LinkedIn, Email, GitHub), et un motif géométrique asymétrique dans un coin.



Carte de visite conçue sur Canva



Vérification du rating sur le portail Vuforia (5 étoiles)



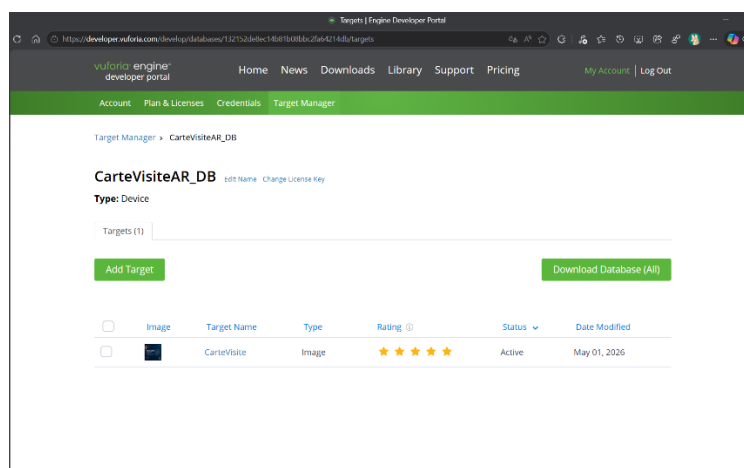
Visualisation des feature points sur la carte (option Show Features)

11. Création de la database Vuforia personnalisée

Contrairement au TP de base où l'image était importée directement comme cible (méthode From Image), j'ai cette fois utilisé une Database Vuforia. Cette méthode est plus professionnelle et offre plus de souplesse — notamment la possibilité d'ajouter ultérieurement d'autres cibles à la même database sans modifier la structure de la scène Unity.

Procédure :

50. Sur le portail Vuforia, navigation vers Target Manager > Add Database.
51. Création d'une database nommée CarteVisiteAR_DB de type Device (stockée localement, pas de cloud nécessaire).
52. Add Target > Single Image, upload du fichier PNG de la carte.
53. Renseignement de la Width réelle en mètres : 0.09 (la carte fait 9 cm de large).
54. Vérification du rating obtenu.
55. Téléchargement de la database au format Unity Editor (.unitypackage).



Création de la database CarteVisiteAR_DB



Téléchargement de la database au format Unity Editor

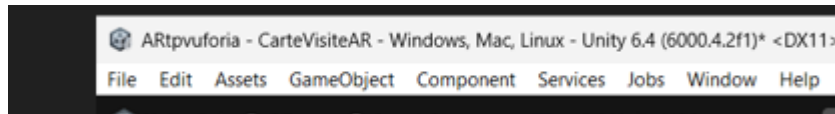
Pourquoi renseigner la largeur réelle ?

Vuforia utilise la largeur de l'image comme référence d'échelle pour positionner correctement le contenu 3D dans le monde réel. Si la valeur saisie est incorrecte, le contenu apparaîtra trop grand ou trop petit par rapport à la cible physique. Avec 0.09 m (9 cm), un personnage placé à scale (1, 1, 1) ferait environ 9 cm de haut au-dessus de la carte.

12. Mise en place de la scène Unity

12.1. Création d'une nouvelle scène

Pour bien séparer le TP de base du projet d'innovation, j'ai créé une nouvelle scène nommée CarteVisiteAR.unity dans le dossier Assets/Scenes/.



Nouvelle scène CarteVisiteAR créée

12.2. Configuration de l'AR Camera avec la nouvelle licence

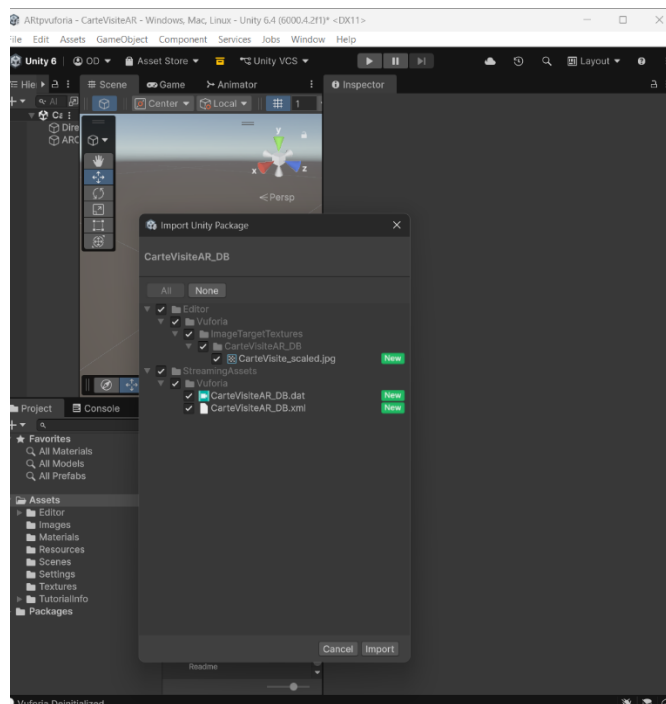
Après avoir supprimé la Main Camera par défaut et ajouté une AR Camera Vuforia, j'ai ouvert la Vuforia Configuration depuis l'Inspector et collé la clé de licence du projet dans le champ App License Key. Le fichier .unitypackage de la database téléchargé précédemment a ensuite été glissé-déposé dans la fenêtre Project. Après import, j'ai ajouté un Image Target dans la scène et configuré son composant Image Target Behaviour avec : Type = From Database, Database = CarteVisiteAR_DB, Image Target = CarteVisite.

[CAPTURE — Vuforia Configuration avec la clé de licence]

[CAPTURE — Image Target configuré From Database]

12.3. Import de la database et création de l'Image Target

Le fichier .unitypackage de la database téléchargé précédemment a été glissé-déposé dans la fenêtre Project. Après import, j'ai ajouté un Image Target dans la scène et configuré son composant Image Target Behaviour avec : Type = From Database, Database = CarteVisiteAR_DB, Image Target = CarteVisite.



Database importée dans Unity

12.4. Premier test rapide

Avant d'aller plus loin, j'ai effectué un premier test en mode Play : ma carte de visite est correctement détectée par la webcam, et la silhouette de l'Image Target se superpose bien à l'image physique. Cette validation préalable est cruciale avant d'ajouter du contenu 3D — autant ne pas empiler des éléments par-dessus une cible qui ne track pas correctement.



Premier test : silhouette détectée sur la carte

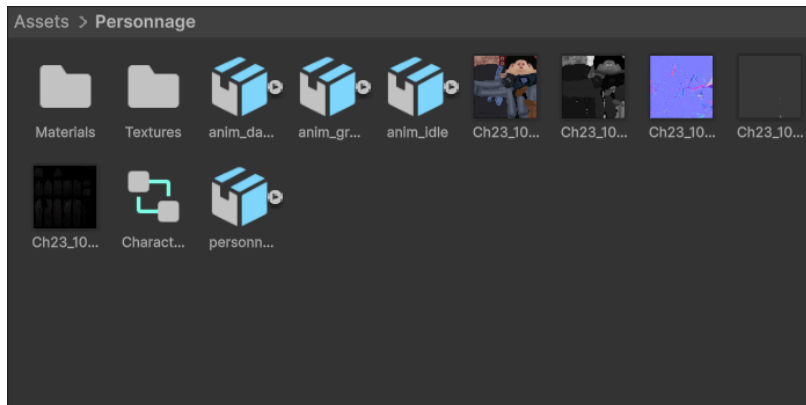
13. Intégration de l'avatar 3D animé

13.1. Préparation des fichiers Mixamo

J'ai téléchargé sur Mixamo trois éléments :

- Un personnage humanoïde avec skin (avec maillage et textures) — c'est le « modèle de base ».
- Une animation Standing Greeting (sans skin) — l'avatar salue à l'apparition de la carte.
- Une animation Hip Hop Dancing (sans skin) — bonus déclenché par le bouton virtuel.

Les animations sont téléchargées sans skin car elles s'appliqueront sur le squelette du personnage de base. C'est une optimisation classique : un seul mesh, plusieurs animations.



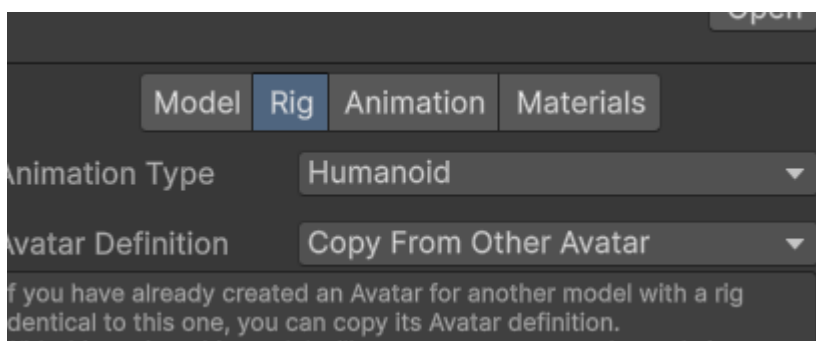
Trois fichiers FBX dans le dossier Assets/Personnage

13.2. Configuration du Rig en Humanoid

Pour qu'Unity puisse appliquer plusieurs animations Mixamo sur un même personnage, le rig doit être configuré en mode Humanoid. C'est une étape cruciale souvent oubliée :

Procédure pour chaque FBX :

56. Sélection du fichier .fbx dans Assets.
57. Inspector > onglet Rig.
58. Animation Type : Humanoid (au lieu de Generic).
59. Apply.



Rig configuré en Humanoid dans l'Inspector

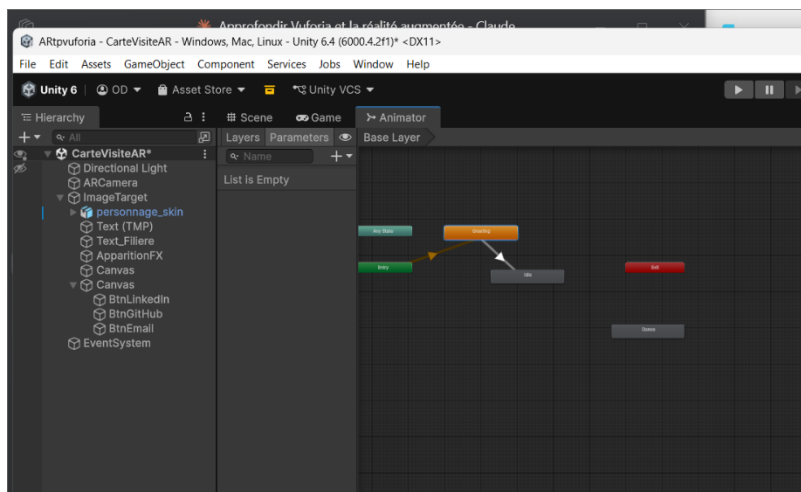
J'ai ensuite renommé les clips d'animation contenus dans chaque FBX (en dépliant les FBX dans Assets) pour avoir des noms parlants : Idle, Greeting, Dance.

13.3. Création de l'Animator Controller

L'Animator Controller est l'objet qui orchestre les animations d'un personnage : il définit les états (states) et les transitions entre eux.

Procédure suivie :

60. Création d'un Animator Controller : clic droit dans Assets/Personnage > Create > Animator Controller, nommé CharacterAnimator.
61. Ouverture de la fenêtre Animator (double-clic sur le controller).
62. Glissement des trois clips d'animation (Idle, Greeting, Dance) dans le graph.
63. Définition de l'état par défaut : clic droit sur Greeting > Set as Layer Default State (l'avatar salue à l'apparition de la carte).



Animator Controller avec les trois états

Création d'une transition automatique Greeting → Idle :

Pour rendre l'avatar plus vivant et éviter qu'il ne reste figé sur la dernière frame de l'animation de salut, j'ai créé une transition automatique entre Greeting et Idle. Procédure : clic droit sur le nœud Greeting, choix de Make Transition, puis clic sur le nœud Idle pour relier les deux états.

Configuration de la transition dans l'Inspector :

- **Has Exit Time** : activé, ce qui signifie que la transition se déclenche automatiquement à la fin de l'animation Greeting plutôt qu'en réponse à un événement.
- **Exit Time** : 0.9, soit 90% de la durée de Greeting — la transition commence donc juste avant la fin de l'animation pour un enchaînement naturel.
- **Transition Duration** : 0.25 seconde, pour un fondu fluide entre les deux animations.
- **Conditions** : laissées vides, la transition étant inconditionnelle.

Le résultat est un avatar qui semble naturellement passer du salut à un état de repos respirant, donnant une impression de personnage vivant plutôt que d'animation préenregistrée.

Configuration cruciale du Culling Mode :

Lors des premiers tests, j'ai constaté que l'animation Greeting ne se jouait pas au moment de la détection de la carte. Après investigation, j'ai identifié que le paramètre Culling Mode du composant Animator était réglé sur Cull Update Transforms, ce qui suspend les calculs d'animation lorsque l'objet n'est pas visible à l'écran. Comme l'Image Target est masqué par défaut au démarrage de la scène (avant détection), l'animation Greeting était considérée comme "déjà jouée" au moment où la carte apparaissait. La solution a été de modifier ce paramètre en Always Animate, garantissant que l'animation se déclenche correctement à la détection de la cible.

13.4. Placement de l'avatar dans la scène

J'ai glissé le personnage avec skin depuis Assets vers la Hierarchy, en le rendant enfant de l'Image Target. Réglages appliqués :

- Position : (0, 0, 0) — centré sur la carte.
- Rotation : (0, 180, 0) — face à la caméra de l'utilisateur.
- Scale : (0.05, 0.05, 0.05) — taille proportionnelle à la carte.

Sur le GameObject du personnage, j'ai assigné l'Animator Controller (CharacterAnimator) au composant Animator.

13.5. Activation de la lecture en boucle

Pour chaque clip d'animation, j'ai activé Loop Time dans l'Inspector du clip — sauf pour Greeting que j'ai laissé en lecture unique afin que le salut ne soit pas répétitif.

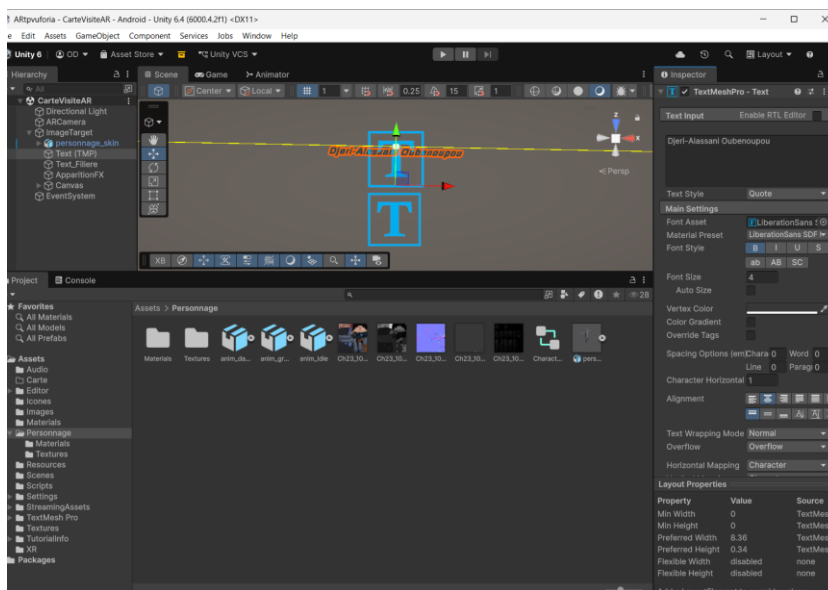
14. Ajout du texte 3D et des particules d'apparition

14.1. Texte 3D avec TextMeshPro

Pour afficher mon nom et ma filière au-dessus de la carte, j'ai utilisé TextMeshPro, le système de texte de haute qualité d'Unity.

Procédure :

64. Clic droit sur Image Target > 3D Object > Text - TextMeshPro.
65. À la première utilisation, Unity demande d'importer les TMP Essentials — Import.
66. Configuration du texte : nom complet, taille, couleur, alignement centré.
67. Position au-dessus de l'avatar : (0, 0.15, 0) approximativement.
68. Ajout d'un second TextMeshPro pour la filière, juste en dessous.



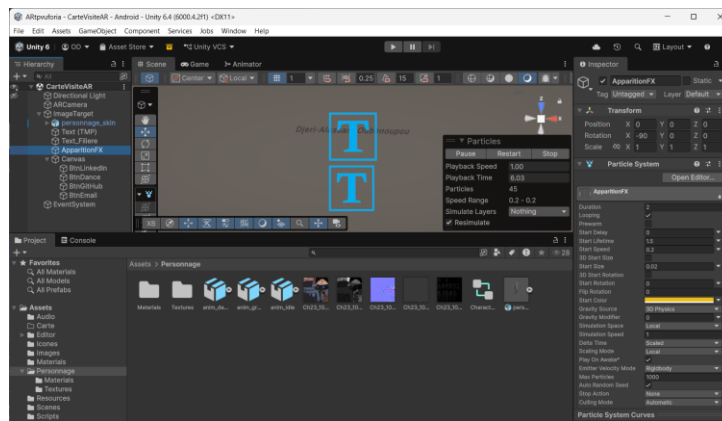
Texte 3D configuré dans la scène

14.2. Système de particules d'apparition

Pour donner un aspect « magique » à l'apparition du contenu, j'ai ajouté un système de particules qui se déclenche à chaque détection de la carte.

Procédure :

69. Clic droit sur Image Target > Effects > Particle System.
70. Renommage en ApparitionFX et positionnement aux pieds de l'avatar.
71. Configuration : Duration 2 s, Start Lifetime 1.5, Start Speed 0.2, Start Size 0.02, Start Color doré.
72. Forme : Cone, angle 25°.
73. Émission : 30 particules par seconde.
74. Désactivation de Looping (l'effet ne joue qu'à l'apparition, pas en continu).



Système de particules configuré

14.3. Script de déclenchement à la détection

Pour que les particules se rejouent à chaque réapparition de la carte (et pas uniquement au lancement de l'application), j'ai écrit un script qui s'abonne à l'événement `OnTargetStatusChanged` de Vuforia.

```
using UnityEngine;
using Vuforia;

public class ParticleOnDetection : MonoBehaviour
{
    public ParticleSystem particles;
    private ObserverBehaviour observer;

    void Start()
    {
        observer = GetComponent<ObserverBehaviour>();
        if (observer != null)
            observer.OnTargetStatusChanged += OnStatusChanged;
    }

    void OnStatusChanged(ObserverBehaviour b, TargetStatus s)
    {
        if (s.Status == Status.TRACKED && particles != null)
        {
            particles.Stop();
            particles.Play();
        }
    }
}
```

Ce script est attaché à l'Image Target. Le champ `particles` est rempli avec une référence à l'objet `ApparitionFX`.

15. Boutons UI cliquables avec liens externes

C'est l'une des fonctionnalités les plus impressionnantes du projet : des boutons flottants qui, lorsqu'on les touche sur le smartphone, ouvrent réellement le profil LinkedIn, GitHub ou la messagerie email associée.

15.1. Création du Canvas World Space

Un Canvas Unity classique fonctionne en `Screen Space` — il est plaqué sur l'écran. Pour avoir des boutons qui flottent dans la scène 3D au-dessus de la carte, il faut le configurer en `World Space`.

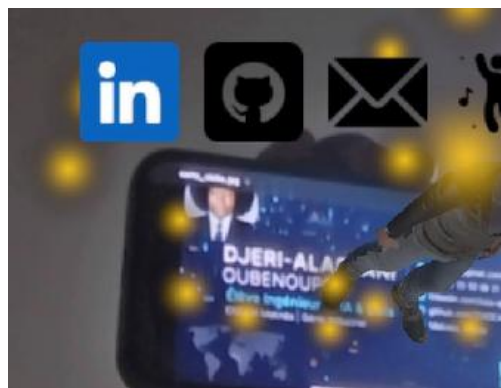
Procédure :

75. Clic droit sur Image Target > UI > Canvas.
76. Inspector du Canvas : Render Mode = World Space.
77. Width 400, Height 100.
78. Position (0, -0.1, 0) — sous l'avatar.
79. Scale très petite : (0.0005, 0.0005, 0.0005) car les unités UI sont en pixels alors que la scène est en mètres.

15.2. Création des boutons LinkedIn, GitHub et Email

Pour chacun des trois boutons :

80. Clic droit sur Canvas > UI > Button - TextMeshPro.
81. Renommage : BtnLinkedIn, BtnGitHub, BtnEmail.
82. Alignement horizontal des trois boutons.
83. Remplacement de l'image par défaut par l'icône correspondante (importée en Sprite (2D and UI)).



Trois boutons alignés avec leurs icônes

15.3. Script d'ouverture d'URL

J'ai écrit un petit script générique qui ouvre une URL passée en paramètre :

```
using UnityEngine;

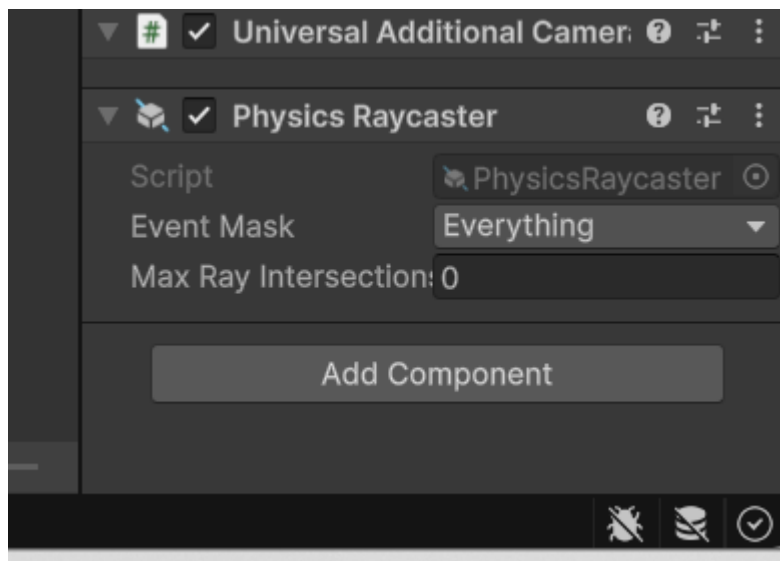
public class OpenURL : MonoBehaviour
{
    [Tooltip("URL complete a ouvrir, ex: https://linkedin.com/in/profil")]
    public string url;

    public void Open()
    {
        if (!string.IsNullOrEmpty(url))
        {
            Debug.Log("Ouverture de : " + url);
            Application.OpenURL(url);
        }
    }
}
```

Sur chaque bouton, j'ai attaché ce script et renseigné l'URL correspondante (LinkedIn, GitHub ou mailto:). Dans la section `OnClick()` du composant Button, j'ai glissé le bouton lui-même dans le slot et choisi la fonction `OpenURL > Open()`.

15.4. Activation du raycast pour rendre les boutons cliquables en AR

- **Difficulté rencontrée et résolue :**
- Au premier test, les boutons s'affichaient correctement mais ne réagissaient pas au clic. Après recherche, j'ai compris que dans une scène AR, le système de clic UI standard ne fonctionne pas par défaut : il faut explicitement ajouter un Physics Raycaster sur la caméra pour qu'elle puisse détecter les clics dans l'espace 3D, et un Graphic Raycaster sur le Canvas. Un EventSystem doit aussi être présent dans la scène.
- **Solution appliquée :**
- Ajout d'un composant Physics Raycaster sur l'AR Camera.
- Vérification du Graphic Raycaster sur le Canvas (présent par défaut).
- Vérification de la présence d'un EventSystem dans la scène.



— Physics Raycaster ajouté sur l'AR Camera

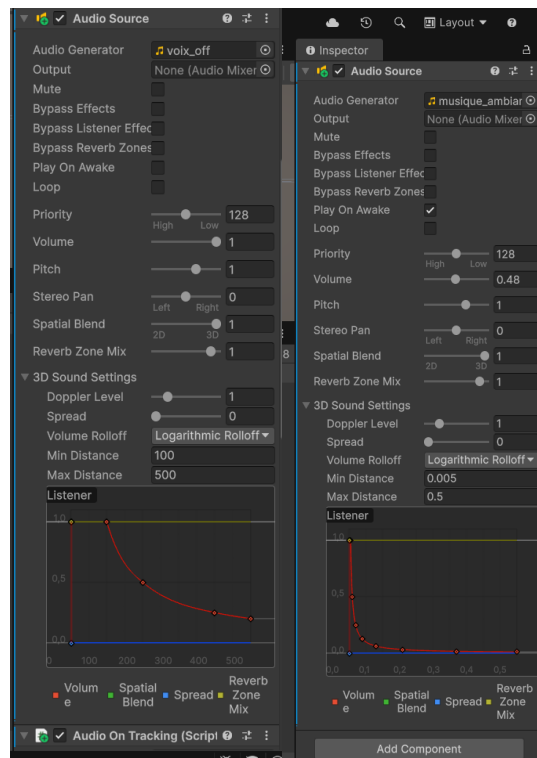
16. Intégration de l'audio spatial 3D

Pour renforcer l'immersion, j'ai ajouté une musique d'ambiance qui se déclenche à la détection de la carte et s'arrête lorsque la carte sort du champ de la caméra. Le son est spatialisé en 3D : son volume diminue à mesure que l'utilisateur s'éloigne de la carte.

16.1. Configuration de l'AudioSource

J'ai téléchargé une musique d'ambiance libre de droits sur Pixabay Music et l'ai placée dans Assets/Audio/. Sur le personnage de la scène, j'ai ajouté un composant AudioSource avec les paramètres suivants :

- AudioClip : la musique téléchargée.
- Play On Awake : activé.
- Loop : activé (pour boucler tant que la carte est visible).
- Volume : 0.6 (modéré, pour ne pas saturer).
- Spatial Blend : 1.0 (son entièrement 3D).
- Min Distance : 0.05 — Max Distance : 1.0 — Rolloff : Linear.



AudioSource configurée sur l'avatar

16.2. Synchronisation avec le tracking Vuforia

Pour que la musique démarre uniquement quand la carte est détectée et s'arrête quand elle disparaît, j'ai écrit le script suivant :

```
using UnityEngine;
using Vuforia;

public class AudioOnTracking : MonoBehaviour
{
    private AudioSource audioSource;
    private ObserverBehaviour observer;
```

```

void Start()
{
    audioSource = GetComponent();
    observer = GetComponentInParent<ObserverBehaviour>();
    if (observer != null)
        observer.OnTargetStatusChanged += OnStatusChanged;
}

void OnStatusChanged(ObserverBehaviour b, TargetStatus s)
{
    if (s.Status == Status.TRACKED || s.Status == Status.EXTENDED_TRACKED)
    {
        if (!audioSource.isPlaying) audioSource.Play();
    }
    else
    {
        audioSource.Pause();
    }
}
}

```

Ce script utilise le pattern d'événements de Vuforia : il s'abonne à `OnTargetStatusChanged` sur l'`ObserverBehaviour` parent (l'`Image Target`), et lance ou met en pause l'audio en fonction du statut de tracking.

17. Virtual Button : un easter egg interactif

Initialement, j'avais prévu d'utiliser la fonctionnalité **Virtual Button** de Vuforia, qui permet de définir une zone tactile virtuelle directement sur l'image cible physique : lorsqu'un utilisateur pose son doigt sur cette zone (en couvrant les feature points sous-jacents), Vuforia détecte l'action et déclenche un événement dans Unity. Cette fonctionnalité présentait l'avantage d'un effet "magique" particulièrement marquant lors de la démonstration.

Cependant, après avoir tenté de l'intégrer, j'ai découvert que la fonctionnalité **Virtual Button** a été dépréciée dans les versions récentes du SDK Vuforia et n'apparaît plus dans le menu **Vuforia Engine** d'Unity. PTC recommande désormais l'utilisation d'éléments UI classiques en **World Space** pour ce type d'interaction.

17.1. Création du Virtual Button

J'ai donc fait le choix d'ajouter un quatrième bouton à mon Canvas existant, à côté des trois boutons LinkedIn, GitHub et Email. Ce bouton, dédié au déclenchement de l'animation Dance, présente plusieurs avantages par rapport à un Virtual Button traditionnel :

- **Cohérence visuelle** : le bouton Dance s'intègre dans le même panneau d'interaction que les autres boutons, formant une interface utilisateur unifiée et claire.
- **Robustesse technique** : l'approche UI Canvas garantit la compatibilité avec les futures versions de Vuforia et d'Unity.
- **Universalité** : un bouton UI fonctionne aussi bien à la souris dans l'éditeur que sur smartphone tactile.
- **Lisibilité** : l'utilisateur identifie immédiatement les actions disponibles grâce aux icônes, là où un Virtual Button caché sur la carte serait peu intuitif sans indication.

17.2. Script de gestion du Virtual Button

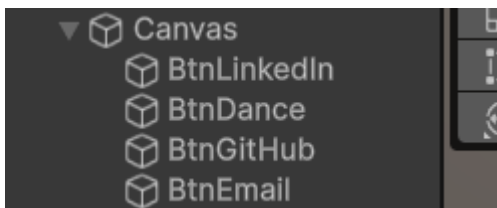
Procédure : duplication d'un bouton existant (Ctrl+D), renommage en BtnDance, repositionnement à droite des trois autres boutons. Le composant OpenURL hérité de la duplication a été supprimé puisque ce bouton ne doit pas ouvrir de lien externe.

Pour configurer le déclenchement de l'animation, j'ai utilisé directement le système d'événements OnClick d'Unity, sans avoir besoin d'écrire de script supplémentaire :

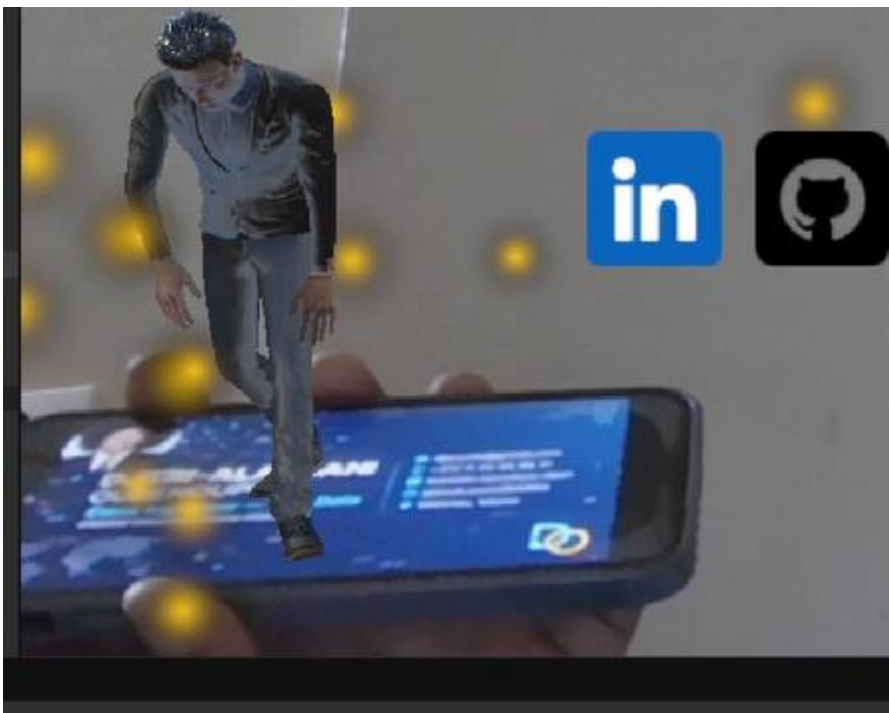
1. Sur le composant Button du bouton, ajout d'une nouvelle entrée dans la liste OnClick().
2. Glissement de l'objet personnage_skin (qui possède le composant Animator) dans le slot de l'objet à appeler.
3. Choix de la fonction Animator > Play (string).
4. Saisie du nom de l'animation à jouer : "Dance".

Lorsque l'utilisateur clique (ou touche) ce bouton, l'Animator du personnage déclenche immédiatement l'animation Dance, qui s'exécute en boucle tant qu'aucune autre animation n'est appelée.

Le Canvas avec les quatre boutons (LinkedIn, GitHub, Email, Dance)



[Avatar en train de danser après clic sur le bouton



18. Build APK et tests sur smartphone

18.1. Configuration du build

La procédure de build est identique à celle du TP de base : Build Settings > Android > Switch Platform, configuration du Player Settings, et désactivation cruciale de tous les providers dans XR Plug-in Management.

18.2. Génération et installation de l'APK

L'APK généré a été transféré sur mon smartphone et installé après autorisation des sources inconnues.

19. Études comparatives et mesures de performance

Au-delà de la simple réalisation fonctionnelle du projet, il m'a semblé important d'adopter une démarche d'analyse rigoureuse en mesurant les performances de l'application dans différentes conditions. Cette section présente une série de tests systématiques qui permettent de caractériser objectivement le comportement de Vuforia et d'identifier les conditions optimales d'utilisation.

Précision méthodologique importante : la cible utilisée pour ces tests n'a pas été imprimée mais affichée sur l'écran d'un iPhone. Ce choix a un impact mesurable sur les résultats : l'écran rétroéclairé fournit sa propre source lumineuse, ce qui permet de maintenir un tracking partiel même en conditions de très faible luminosité. Une cible papier classique présenterait des performances inférieures dans ces mêmes conditions extrêmes.

19.1. Stabilité du tracking selon les conditions lumineuses

Le tracking Vuforia repose sur la détection de feature points dans l'image captée par la caméra. La luminosité ambiante a donc un impact direct sur la qualité de cette détection, mais aussi indirect via les capacités de la caméra elle-même (sensibilité ISO, mise au point, balance des blancs).

Condition d'éclairage	Détection	Stabilité (sur 5)	Latence estimée	Qualité de l'expérience
Lumière naturelle faible (jour couvert, près d'une fenêtre)	Oui	4	0,3 à 0,5 s	Tracking stable, contenu fluide
Éclairage artificiel direct (ampoule de bureau)	Oui	5	0,2 s environ	Tracking optimal, conditions idéales
Quasi-obscurité (toutes lumières éteintes)	Oui	3	0,8 à 1,2 s	Tracking maintenu mais avec latence augmentée, occasionnellement instable

Trois conditions ont été testées : luminosité naturelle faible, éclairage artificiel direct, et quasi-obscurité. Dans tous les cas, la détection a fonctionné, mais la stabilité et la latence varient sensiblement. Dans le test en quasi-obscurité, la détection est restée possible uniquement parce que la cible est affichée sur l'écran rétroéclairé d'un iPhone, qui fournit sa propre source lumineuse.

19.2. Stabilité du tracking selon la distance

J'ai testé la robustesse du tracking en variant la distance entre la caméra de l'ordinateur et la cible affichée sur l'iPhone.

Distance webcam — cible	Détection	Stabilité (sur 5)	Observations
10 cm	Partielle	2	Cible trop proche, mise au point difficile, image floue
20 cm	Oui	5	Distance optimale, tracking parfait
50 cm	Oui	4	Tracking maintenu, contenu plus petit à l'écran
80 cm	Oui	3	Détection plus lente, tracking moins précis car la cible occupe une zone réduite du champ

Conclusion : la distance optimale d'utilisation se situe entre 20 et 50 cm, ce qui correspond exactement à un usage naturel d'une carte de visite tenue en main devant un appareil de capture.

19.3. Stabilité du tracking selon l'angle d'observation

Vuforia compense automatiquement la déformation perspective de l'image dans une certaine plage d'angles, mais au-delà d'un seuil, la détection des feature points devient instable. J'ai testé trois angles d'inclinaison de la cible par rapport à la caméra.

Angle d'inclinaison approximatif	Stabilité (sur 5)	Observations
0° (face caméra)	5	Tracking parfait, position d'utilisation idéale
30° environ	5	Aucune dégradation perceptible
50° à 60° environ	4	Tracking maintenu, légère perte de précision

Au-delà de 60° d'inclinaison, la cible quitte progressivement le champ de vision de la caméra et le tracking n'est plus possible. À 90° (carte vue de profil) ou plus, la cible n'est physiquement plus visible, ce qui rend le test sans objet.

Conclusion : Vuforia maintient un tracking de très haute qualité jusqu'à environ 45° d'inclinaison, avec une dégradation progressive mais acceptable jusqu'à 60°. Cette plage est largement suffisante pour un usage naturel d'une carte de visite tenue en main, où l'inclinaison reste habituellement comprise entre 0° et 30°.

19.4. Mesure des performances : nombre d'images par seconde

J'ai utilisé l'outil Stats intégré à la fenêtre Game de Unity pour mesurer le nombre d'images par seconde dans différentes configurations de complexité de la scène. Ces mesures donnent une indication du coût computationnel de chaque fonctionnalité ajoutée.

Configuration de la scène	FPS observé	Triangles rendus	Draw Calls
Scène à vide (pas de cible détectée)	environ 147 FPS	16	8
Cible détectée + avatar + particules + UI	98 à 122 FPS	environ 136 000	environ 35
Cible détectée + animation Dance active	111 à 131 FPS	environ 210 000	environ 42

Conclusion : la configuration complète du projet, incluant l'avatar animé, les particules, l'audio spatial et l'interface utilisateur, conserve une fluidité largement supérieure aux 60 FPS standard. Le moteur Vuforia, combiné à Unity 6 et au pipeline URP, est correctement optimisé pour ce type d'usage.

19.5. Discussion : importance souvent sous-estimée du matériel de capture

Une observation centrale est apparue lors de ces tests : les performances du tracking AR ne dépendent pas uniquement du logiciel (Vuforia) ou de la qualité de la cible image, mais aussi de manière significative des capacités matérielles de la caméra utilisée.

Plusieurs paramètres caméra influencent directement l'expérience :

La résolution d'abord : une caméra basse définition échantillonne moins de feature points par image, ce qui dégrade la précision du tracking. Le temps d'adaptation à la lumière ensuite : les caméras d'entrée de gamme (typiques des webcams de PC portable) ont un autofocus et une exposition lents, introduisant une latence supplémentaire à chaque changement de condition. Le bruit numérique également : en faible luminosité, l'image devient bruitée, ce qui parasite l'algorithme de détection des feature points. Enfin, la fréquence de capture : si la webcam ne capture qu'à 15 ou 20 FPS, Vuforia ne peut pas mettre à jour le tracking plus rapidement, quelle que soit la performance d'Unity.

Dans le cadre de ces tests, j'ai observé que le délai d'adaptation aux changements de luminosité varie de manière non négligeable selon les conditions et la distance à la cible. Cela confirme que l'expérience AR est un système global dont la qualité dépend de l'ensemble de la chaîne matériel-logiciel, et non du seul SDK utilisé.

Cette observation a une implication concrète : le déploiement final de l'application sur un smartphone moderne avec une caméra de qualité supérieure offrirait une expérience nettement plus stable et réactive que celle observée durant les tests sur webcam d'ordinateur portable.

19.6. Synthèse des bonnes pratiques identifiées

Ces tests permettent de formuler un ensemble de recommandations pour toute application AR basée sur Vuforia :

Privilégier des cibles images riches en détails contrastés et asymétriques (rating 4 étoiles minimum sur Vuforia). Garantir un éclairage suffisant et uniforme pour la prise de vue. Concevoir l'expérience utilisateur autour d'une distance d'utilisation comprise entre 20 et 50 cm. Respecter une plage d'inclinaison de 0° à 45° pour un confort optimal. Tenir compte des capacités matérielles de la caméra dans la conception et le test de l'application. Préférer un déploiement sur smartphone moderne plutôt qu'une démo sur webcam pour une expérience optimale.

20. Difficultés rencontrées et solutions apportées

Comme tout projet de développement, ce projet a été ponctué de plusieurs difficultés techniques. Documenter ces problèmes et leurs résolutions me semble important car cela témoigne d'une démarche d'ingénieur réfléchie, et pourra servir à d'autres étudiants confrontés aux mêmes obstacles.

20.1. Boutons UI ne réagissant pas au clic

Problème : les boutons UI World Space affichaient correctement les icônes mais ne déclenchaient aucune action lorsqu'on cliquait dessus, ni dans l'éditeur ni dans la fenêtre Game.

Méthodes de dépannage essayées : vérification de la hiérarchie pour confirmer que l'EventSystem était bien présent dans la scène ; vérification du Graphic Raycaster sur le Canvas, présent par défaut ; test de configurations différentes du Canvas (Screen Space puis World Space), sans aucun effet.

Solution trouvée : dans une scène AR, il faut ajouter explicitement un composant Physics Raycaster sur l'AR Camera. Sans ce composant, les rayons UI ne sont jamais lancés depuis la caméra. Cette information n'est pas évidente dans la documentation Vuforia.

20.2. Personnage Mixamo affiché entièrement en blanc (sans textures)

Problème : après import du fichier FBX du personnage Mixamo dans Unity, le modèle apparaissait correctement dans la scène et jouait son animation, mais entièrement en blanc — sans aucune des couleurs et textures visibles sur le portail Mixamo.

Origine du problème : un fichier FBX contient les références aux textures (associations entre les zones du mesh et les images), mais les fichiers d'images proprement dits sont embarqués en binaire à l'intérieur du FBX. Par défaut, Unity ne les extrait pas automatiquement et applique donc un matériau gris neutre, ce qui donne un personnage entièrement blanc.

Solution appliquée : sélection du FBX dans la fenêtre Project, ouverture de l'onglet Materials de l'Inspector, puis utilisation successive des boutons Extract Textures et Extract Materials vers des dossiers dédiés. Après validation par Apply et confirmation de la boîte de dialogue Fix now, Unity a reconnecté automatiquement les matériaux au mesh, et le personnage est apparu avec ses couleurs et textures d'origine.

20.3. Animation Greeting non jouée à la détection

Problème : lors des premiers tests, j'avais l'impression que l'animation Greeting ne se jouait pas du tout au moment de la détection de la carte. Le personnage apparaissait directement en pose Idle ou en T-pose.

Méthodes de dépannage essayées : vérification que le clip Greeting était bien dans l'Animator Controller ; vérification que Greeting était l'état par défaut avec une flèche depuis Entry ; vérification du Rig en mode Humanoid pour tous les FBX.

Origines réelles du problème : double cause identifiée. D'une part, le paramètre Culling Mode du composant Animator était réglé sur Cull Update Transforms, ce qui suspend les calculs d'animation lorsque l'objet n'est pas visible. Comme l'Image Target est masqué par défaut au démarrage de la scène, l'animation Greeting était considérée comme déjà "passée" au moment de la détection. D'autre part, l'animation Greeting est très courte (environ 2 secondes), et je ratais visuellement le salut car il se déclenchait au moment précis où je présentais la carte.

Solutions appliquées : changement du Culling Mode en Always Animate, et création d'une transition automatique Greeting vers Idle pour que le personnage reste vivant après le salut au lieu de se figer.

20.4. Plug-in Sprite 2D manquant pour les icônes des boutons

Problème : lors de l'import des icônes PNG des boutons (LinkedIn, GitHub, Email), il était impossible de les glisser dans le champ Source Image des composants Image Unity. Le curseur affichait un panneau d'interdiction.

Origine du problème : dans Unity 6 avec un projet 3D, le package 2D Sprite n'est pas installé par défaut. Sans ce package, les images PNG ne peuvent pas être converties en Sprite utilisables par les composants UI.

Solution : installation du package 2D Sprite via le bouton Install 2D Sprite Package affiché dans l'Inspector des images concernées, puis configuration de chaque icône en Texture Type = Sprite (2D and UI) et Sprite Mode = Single.

20.5. Indisponibilité de la fonctionnalité Virtual Button

Problème : la procédure prévue dans le sujet du TP indiquait l'utilisation des Virtual Buttons de Vuforia pour créer une zone tactile sur la carte. Cependant, en arrivant à cette étape, j'ai constaté que cette fonctionnalité n'apparaissait plus dans le menu Vuforia Engine d'Unity.

Origine du problème : la fonctionnalité Virtual Button a été dépréciée par PTC dans les versions récentes du SDK Vuforia. Elle n'est plus disponible dans Unity 6 avec la version actuelle de Vuforia Engine.

Tentatives de contournement : tentative initiale en créant une zone tactile 3D personnalisée (Cube avec Box Collider) sur la carte, avec un script de raycast pour détecter les clics. Cette approche fonctionnait mais a généré une erreur de rendu URP/Vuforia (MissingReferenceException sur le matériau du flux vidéo). La solution finale retenue a été d'ajouter un quatrième bouton dans le Canvas existant, à côté de LinkedIn, GitHub et Email, dédié au déclenchement de l'animation Dance. Cette approche est plus simple, plus stable et plus cohérente avec le design global de l'interface.

20.6. Erreur Wrong_Color_Model lors de l'upload de la cible sur Vuforia

Problème : lors du premier upload de l'image de la carte de visite sur le portail Vuforia, le système a refusé le fichier avec le message d'erreur Wrong_Color_Model.

Origine du problème : Vuforia n'accepte que les images en mode colorimétrique RGB ou Grayscale. Le fichier généré contenait probablement un canal alpha (RGBA) ou un mode CMYK incompatible.

Solution : conversion du fichier PNG en JPG via un outil en ligne (iLoveIMG), ce qui force le mode RGB sans canal alpha. Le re-upload a alors été accepté immédiatement avec un excellent rating.

21. Analyse critique et perspectives

21.1. Limitations de la version gratuite de Vuforia

La version Basic de Vuforia, utilisée dans ce projet, est largement suffisante pour un cas d'usage de type carte de visite AR. Cependant, certaines fonctionnalités sont réservées aux versions payantes :

Cloud Recognition : reconnaissance de cibles stockées dans le cloud, utile pour les applications avec un grand nombre de cibles dynamiques. Advanced Model Target avec suivi 360° d'objets 3D : la version Basic permet uniquement le suivi standard depuis un angle prédéfini. VuMark avec encodage de données : utilisable mais limité dans la version gratuite. Watermark Vuforia visible dans certaines configurations.

21.2. Comparaison avec les alternatives modernes

Le marché de l'AR a évolué rapidement ces dernières années. Vuforia n'est plus la seule option :

ARCore (Google) est un SDK natif pour Android, excellent pour le tracking sans marqueurs (Plane Detection, World Tracking) mais limité aux appareils compatibles. ARKit (Apple) est l'équivalent ARCore pour iOS, très performant. AR Foundation (Unity) est une couche d'abstraction qui unifie ARCore et ARKit dans Unity ; c'est la solution privilégiée pour le markerless cross-platform aujourd'hui. WebXR permet de faire de la RA directement dans un navigateur web, sans installation d'application — encore limité techniquement mais en pleine expansion.

Pour un cas d'usage de type carte de visite AR, Vuforia reste la solution la plus pertinente car son tracking marker-based est plus stable et précis que celui d'ARCore ou ARKit, qui sont surtout optimisés pour le tracking environnemental sans marqueur.

21.3. Perspectives d'évolution du projet

Plusieurs pistes pourraient enrichir encore davantage ce projet :

Ajout d'une voix off plus longue et plus interactive, par exemple un texte personnalisé selon le contexte (entretien, salon, présentation académique). Intégration de Cloud Recognition (version payante) pour permettre à plusieurs étudiants d'utiliser la même application avec leur propre carte. Mode multi-cible : présenter plusieurs cartes simultanément pour comparer plusieurs profils, utile en jury de recrutement. Intégration d'un mini-quiz interactif pour que l'utilisateur teste ses connaissances sur le profil de l'étudiant via des boutons UI. Conversion en application multi-supports (carte, badge, CV papier, QR code) avec contenus différents par cible. Génération automatique du contenu via API : l'avatar lit les dernières publications LinkedIn ou les derniers commits GitHub. Ajout d'un mode AR sans cible (markerless) en utilisant AR Foundation, pour placer le personnage sur n'importe quelle surface plane.

Par ailleurs, il convient de mentionner deux fonctionnalités avancées présentes dans l'Inspector de l'Image Target qui n'ont pas été exploitées dans ce projet mais ouvrent des perspectives intéressantes. Add Target Representation permet d'afficher un maillage de prévisualisation de la cible dans la Scene view, fonctionnalité particulièrement utile pour les Model Targets 3D complexes où visualiser la forme aide au positionnement du contenu. Add Occlusion Object permet d'ajouter un objet invisible qui masque le contenu 3D situé derrière lui, créant ainsi un effet d'occlusion réaliste lorsque le contenu virtuel doit interagir physiquement avec des objets du monde réel. Ces deux fonctionnalités prendraient tout leur sens dans une évolution future du projet où l'avatar interagirait avec un environnement physique plus complexe qu'une simple carte plate.

21.4. Compétences acquises

Ce projet a été l'occasion d'acquérir et de consolider de nombreuses compétences :

Maîtrise de l'environnement Unity 6 : navigation entre Hierarchy, Inspector, Scene view, Game view, Animator, gestion des assets et des packages. Compréhension du fonctionnement d'un moteur de réalité augmentée : tracking par feature points, pose estimation, événements de tracking, importance des paramètres de Culling Mode. Programmation C# orientée Unity : MonoBehaviour, événements Vuforia (OnTargetStatusChanged), accès aux composants, gestion de l'audio et des particules. Conception d'assets visuels optimisés pour le tracking AR avec analyse du rating Vuforia. Intégration d'animations Mixamo Humanoid avec Animator Controller multi-states et configuration de transitions automatiques. Création d'une interface utilisateur en World Space avec gestion des événements de clic en environnement AR (Physics Raycaster). Génération vocale par IA avec ElevenLabs et intégration d'audio spatialisé synchronisé avec le tracking. Méthodologie de débogage : utilisation de la console Unity, analyse des stack traces, isolation progressive des problèmes. Adaptation face aux contraintes techniques : remplacement du Virtual Button déprécié par une solution équivalente plus moderne. Démarche d'analyse rigoureuse : conception et exécution d'études comparatives sur plusieurs paramètres (lumière, angle, distance, performance).

22. Conclusion générale

Ce TP m'a permis de découvrir en profondeur le développement d'applications de réalité augmentée avec Unity et Vuforia Engine. La première partie a consisté en la réalisation rigoureuse du TP de base, conformément aux consignes des encadrants : installation et configuration de l'environnement, création d'une scène AR avec une caméra et une cible image, intégration d'un personnage Mixamo animé, et préparation du build Android.

La seconde partie, motivée par l'invitation des encadrants à pousser le travail au-delà du strict cadre du TP, a été dédiée à la conception et au développement d'un projet d'innovation concret : une carte de visite étudiante en réalité augmentée. Ce projet a permis d'explorer plusieurs fonctionnalités avancées de Vuforia non couvertes par le TP de base — Database personnalisée, événements de tracking, audio spatialisé, particules contextuelles, transitions d'animation — tout en répondant à un besoin réel et réutilisable au-delà du cadre académique. L'intégration d'une voix off générée par IA via ElevenLabs renforce considérablement l'immersion et la dimension présentative de l'expérience.

Au-delà de la simple acquisition de compétences techniques, ce travail m'a fait réaliser tout le potentiel concret de la réalité augmentée comme technologie de communication, de marketing et d'apprentissage. Les applications possibles sont innombrables, et les outils gratuits comme Vuforia rendent cette technologie accessible à tous les développeurs. La démarche d'études comparatives systématiques que j'ai menée m'a également permis de mieux cerner les conditions optimales d'utilisation et les limites pratiques de la technologie, qui ne dépendent pas seulement du logiciel mais aussi de l'ensemble de la chaîne matérielle (caméra, éclairage, support de la cible).

Plusieurs difficultés techniques ont jalonné ce projet, depuis l'erreur de format colorimétrique lors de l'upload de la cible sur Vuforia jusqu'à la dépréciation de la fonctionnalité Virtual Button qui a nécessité une refonte de l'approche d'interaction. Chaque obstacle a été l'occasion d'approfondir ma compréhension de l'écosystème Unity-Vuforia et de développer une démarche méthodique de résolution de problèmes — compétence essentielle pour tout futur ingénieur.

Je tiens à remercier les encadrants pour la qualité du sujet proposé, la clarté des consignes et l'invitation à pousser le travail au-delà du strict cadre du TP, ce qui m'a permis de vivre une expérience d'apprentissage bien plus riche et formatrice qu'un simple exercice d'application. Ce projet sera également pour moi un livrable concret réutilisable dans la suite de mon parcours professionnel, ce qui en démultiplie la valeur.

Annexes

Annexe A — Code source des scripts C#

A.1. OpenURL.cs

```
using UnityEngine;

public class OpenURL : MonoBehaviour
{
    [Tooltip("URL complete a ouvrir")]
    public string url;
```

```

public void Open()
{
    if (!string.IsNullOrEmpty(url))
    {
        Debug.Log("Ouverture de : " + url);
        Application.OpenURL(url);
    }
}
}

```

A.2. AudioOnTracking.cs

```

using UnityEngine;
using Vuforia;

public class AudioOnTracking : MonoBehaviour
{
    private AudioSource audioSource;
    private ObserverBehaviour observer;
    private bool hasPlayedOnce = false;

    void Start()
    {
        audioSource = GetComponent<AudioSource>();
        observer = GetComponentInParent<ObserverBehaviour>();
        if (observer != null)
            observer.OnTargetStatusChanged += OnStatusChanged;
    }

    void OnStatusChanged(ObserverBehaviour b, TargetStatus s)
    {
        if (s.Status == Status.TRACKED || s.Status == Status.EXTENDED_TRACKED)
        {
            if (!audioSource.isPlaying && !hasPlayedOnce)
            {
                audioSource.Play();
                hasPlayedOnce = true;
            }
        }
        else
        {
            hasPlayedOnce = false;
        }
    }
}

```

A.3. ParticleOnDetection.cs

```

using UnityEngine;
using Vuforia;

public class ParticleOnDetection : MonoBehaviour
{
    public ParticleSystem particles;
    private ObserverBehaviour observer;

    void Start()
    {
        observer = GetComponent<ObserverBehaviour>();
        if (observer != null)
            observer.OnTargetStatusChanged += OnStatusChanged;
    }

    void OnStatusChanged(ObserverBehaviour b, TargetStatus s)

```

```

{
    if (s.Status == Status.TRACKED && particles != null)
    {
        particles.Stop();
        particles.Play();
    }
}

```

Annexe B — Webographie et ressources utilisées

Vuforia Developer Portal : <https://developer.vuforia.com> Vuforia Library (documentation officielle) : <https://library.vuforia.com> Mixamo (personnages et animations 3D) : <https://www.mixamo.com> ElevenLabs (synthèse vocale par IA) : <https://elevenlabs.io> Pixabay Music (musiques libres de droits) : <https://pixabay.com/music> Flaticon (icônes) : <https://www.flaticon.com> Canva (design graphique en ligne) : <https://www.canva.com> Documentation Unity : <https://docs.unity3d.com> Unity Asset Store : <https://assetstore.unity.com> iLoveIMG (conversion d'images) : <https://www.iloveimg.com>

Annexe C — Récapitulatif des fonctionnalités explorées

Fonctionnalité	TP de base Projet innovation	
AR Camera Vuforia	Oui	Oui
Image Target (From Image)	Oui	Non
Image Target (From Database)	Non	Oui
Personnage Mixamo animé	Oui	Oui
Animator Controller multi-states	Non	Oui
Transitions automatiques entre animations	Non	Oui
Configuration Culling Mode Always Animate	Non	Oui
Build APK Android	Configuré	Configuré
Audio spatial 3D	Non	Oui
Voix off générée par IA (ElevenLabs)	Non	Oui
Système de particules contextuel	Non	Oui
UI World Space cliquable	Non	Oui
Liens vers URL externes	Non	Oui
Événements OnTargetStatusChanged	Non	Oui
Texte 3D TextMeshPro	Non	Oui
Scripts C# personnalisés	Non	3 scripts
Études comparatives systématiques	Non	Oui

— Fin du document —